



# Developers' Dilemma: Opportunities and Pitfalls of Generative AI for Software Development

Jan Zacharias · Alina Popova · Moritz von Zahn · Johannes Chen · Oliver Hinz

Received: 5 September 2025 / Accepted: 3 February 2026  
© The Author(s) 2026

**Abstract** Generative artificial intelligence (GenAI) has attracted worldwide attention across numerous domains, with particularly strong implications for software development. Effective integration of GenAI requires a thorough understanding of both its opportunities and pitfalls for software developers and the organizations they are embedded in. In this study, 30 expert interviews were conducted to examine the sociotechnical impact of GenAI from the perspective of software development practitioners. The findings show that GenAI can enhance individual and organizational productivity and stimulate creative problem-solving. At the same time, concerns emerge around over-reliance, skill degradation, and homogenization among developers. Moreover, with GenAI rapidly providing valuable information, communication between developers may decline, potentially undermining interpersonal relationships and team cohesion. These insights contribute to information systems theory by examining GenAI's effects on social capital, machine-induced reflections, and developers' professional identity, while also offering practical implications for the effective and sustainable adoption of GenAI in software development.

**Keywords** Generative artificial intelligence · Large language models · Coding assistants · Software development · Professional identity

## 1 Introduction

In recent years, generative artificial intelligence (GenAI) has garnered substantial attention from businesses, academia, and the general public (Noy and Zhang 2023; Susarla et al. 2023; Feuerriegel et al. 2024; Hoyek et al. 2024). Pivotal to this development was the release of ChatGPT in late 2022, kicking off a fast series of advancements in the area of GenAI technology. The applications of GenAI are diverse, covering numerous domains, such as finance (Dhake et al. 2024), healthcare (Cascella et al. 2023), or higher education (Jürgensmeier and Skiera 2024).

GenAI systems are having a particularly strong impact on software development, with rapid adoption of tools such as ChatGPT, DeepSeek, Gemini, GitHub Copilot, and Amazon CodeWhisperer. In software development, Information Systems (IS) research has long examined the influence of information technology (IT) artifacts for coding assistance (e.g., Banker and Kauffman 1991; Orlikowski 1993; Stewart and Gosain 2006). IS researchers typically view IT artifacts as passive tools that are controlled by human users with a “primacy to human agency” (Baird and Maruping 2021). With the emergence of GenAI, we are witnessing a fundamental shift in the way IT artifacts influence software development. GenAI-powered IT artifacts act with agentic, almost human-like properties: they can partly infer user intentions, write and run programming code, and autonomously propose solutions that the user may not have considered before (Feuerriegel et al. 2024). This shift challenges the primacy to human agency and changes the relation between humans and IT artifacts from delegation (Baird and Maruping 2021) to co-creation (Feuerriegel et al. 2024). As a result, the role of human software developers changes as they now interact with a digital partner they must both steer and critically validate.

---

Accepted after 2 revisions by Alexander Benlian.

---

J. Zacharias (✉) · A. Popova · M. v. Zahn · J. Chen · O. Hinz  
Faculty of Economics and Business, Goethe University  
Frankfurt, Theodor-W.-Adorno Platz 4,  
60323 Frankfurt am Main, Germany  
e-mail: zacharias@wiwi.uni-frankfurt.de

Against this background, our work explores the broad impact of GenAI systems on both individual software developers and the organizations they are part of.

Previous studies have demonstrated the impact of GenAI on the productivity of developers in controlled experimental settings (e.g., Peng et al. 2023; Li et al. 2024; Wang et al. 2024a), but a broader sociotechnical perspective on its effects remains largely unexplored. Existing research suggests that developers may apply GenAI systems to increase their productivity in all phases of the software development process, that is, from requirements analysis to maintenance and support (Pothukuchi et al. 2023). For instance, in the development phase, GenAI may accelerate the generation of boilerplate code and documentation, while in the testing phase, it generates and automates testing scripts (Deniz et al. 2023; Ebert and Louridas 2023; Pothukuchi et al. 2023). Despite these benefits, little is known about how GenAI adoption influences the broader sociotechnical dynamics of software development. Specifically, the sociotechnical perspective considers not only the technical dimensions of IT adoption – which involves the IT artifacts, processes, and the resulting productivity gains – but also the social dimensions related to individual behavior, collaboration patterns, and social capital (Sarker et al. 2019). The IS research community has yet to adopt a comprehensive sociotechnical perspective on GenAI's effects on software development, a gap that this paper aims to narrow.

A sociotechnical perspective is crucial for understanding the opportunities and pitfalls of adopting new IT in organizations. For example, in healthcare, IT based on artificial intelligence (AI) improves diagnostic accuracy and general operational efficiency but also poses challenges like cybersecurity issues, loss of human control, and the fear of job losses (Lee and Yoon 2021). Similarly, in the field of software development, early evidence suggests that GenAI can also have both a positive and negative impact on individuals and organizations. On the one hand, studies indicate that the rapid efficiency gains make developers happier and more satisfied as they can shift their focus from producing syntax-compliant code to more stimulating tasks (Ebert and Louridas 2023; Chen 2024). On the other hand, there is general concern about GenAI-induced layoffs that is frequently highlighted by the media (Thorbecke 2023; Cerullo 2024). In addition, task automation using AI technologies can alter the self-perception of experts whose work is affected and, eventually, may threaten their professional identity (Jussupow et al. 2022; Mirbabaie et al. 2022). Prior evidence shows that developers' well-being is closely intertwined with their work practices (see, e.g., Benlian et al. 2025), suggesting that their expectations regarding a technology's long-term impacts on both their professional future and their organization's trajectory are

likely to shape their intention to use it (Stein et al. 2015; Bosnjak et al. 2020). Against this backdrop, it is vital for the IS research community to study software developers' perceptions of GenAI and gain a broader, sociotechnical understanding of its impact. Such an understanding is also essential for businesses to integrate GenAI effectively and sustainably into organizational workflows (Madaio et al. 2022).

In this paper, we explore the broad sociotechnical impact of GenAI on both individuals and organizations in the field of software development. Specifically, we investigate the functional affordances (Markus and Silver 2008) through which developers interact with GenAI in practice and analyze the opportunities and pitfalls associated with them, as well as their implications for the professional role of software developers. Thus, we aim to answer the following research questions: *What are the opportunities and pitfalls of GenAI adoption for software developers and their organizations? How will the professional role of software developers evolve in response to GenAI adoption?* To investigate these questions, we conduct a qualitative study involving 30 interviews with practitioners from the field of software development. Our study yields insight on how GenAI affects developers' work environment and captures their attitudes and emotions toward its adoption. Using inductive qualitative analysis, we develop a conceptual model of the impact of GenAI adoption in software development from a sociotechnical perspective.

Our findings reveal that, at the individual level, GenAI systems increase developers' productivity, supports creative problem-solving, and facilitates self-education, but also poses the risk of over-reliance and skill degradation. At the organizational level, GenAI allows teams to handle more projects and improves cross-department communication and general operational efficiency while potentially weakening interpersonal relationships by partially replacing developer interactions. These shifts contribute to a redefinition of developers' roles and may challenge their traditional expertise monopoly.

Our study makes important contributions to theory and practice. By focusing on practitioners' perspectives beyond productivity effects, we contribute to a more holistic view of GenAI's role in software development. Specifically, we highlight the potential harms of GenAI on social capital within businesses, offering a novel perspective on the impact of IT on social capital in organizations (see, e.g., Huysman and Wulf (2006); Weiler et al. (2022)). Additionally, our findings suggest that GenAI may enhance creative problem-solving, providing further evidence to support existing theories on machine-induced reflections (see, e.g., Pfeuffer et al. (2023)). We further observe that developers expect GenAI to bring a shift in their professional identity, aligning with emerging theories of AI as an

identity threat (Jussupow et al. 2022; Mirbabaie et al. 2022). Finally, we add to the ongoing discussion on GenAI in the workplace (Söllner et al. 2025) by identifying long-term risks, such as skill degradation, and hurdles that organizations should overcome to achieve a successful adoption of GenAI.

The paper proceeds as follows. First, we present the relevant background in the literature, followed by our methodology. Next, we outline our analysis plan and present the results of our study. Finally, we discuss our findings and conclude with directions for future research.

## 2 Background

This section presents related work and the relevant literature streams to which our study contributes. First, we outline how our work relates to research on GenAI in software development and how it extends this research by employing a broader sociotechnical view. Second, we make a deep dive into the concept of the sociotechnical perspective in IS. Third, we present background literature on how GenAI may serve as an opportunity or a pitfall for social capital within organizations, machine-induced learning and reflection, and professional identity in the IT sector.

### 2.1 Generative AI in Software Development

Our study focuses on software development, a mostly collaborative profession that relies on creativity, expertise, and extensive communication (Forsgren et al. 2021; Ross et al. 2023). A central framework in this profession is the software development life cycle (SDLC), which typically spans the following phases: requirements analysis, design, development, testing, deployment, and maintenance (Bassil 2012). During the requirements analysis phase, developers gather business requirements and define stakeholder needs for the planned software product. The design phase follows, where developers create a detailed blueprint of the software, including its architecture, data structures, and user interface. In the development phase, developers write the source code, thereby translating the design into a functional system. In the subsequent testing phase, they verify and validate the software to ensure it meets the specified requirements and, if applicable, address any bugs identified. Finally, the deployment and maintenance phases involve delivering the software to the customer and making necessary adjustments to optimize performance in its operational environment.

In recent years, tasks within the SDLC have been augmented by GenAI. GenAI is a type of AI specialized in generating new data in various formats, such as text,

images, and audio (Susarla et al. 2023). Its capabilities are underpinned by a set of distinct properties that fundamentally differentiate it from traditional tools used in software development: its generative, conversational, advanced reasoning, and variability properties (Banh et al. 2025; Storey et al. 2025; Wessel et al. 2025).

While traditional tools operate deterministically, relying on rule-based logic, GenAI possesses a generative property, enabling the creation of novel artifacts not constrained by predefined rules (Feuerriegel et al. 2024). Further, the variability property allows GenAI to generate multiple unique outputs from the same input, thereby supporting exploration and diversity in problem-solving (Banh and Strobel 2023). The conversational property allows interactive, conversational, context-sensitive engagement with users through natural language (Ramaul et al. 2024). Unlike traditional software tools, whose interaction mechanisms are limited in scope and adaptability, GenAI is able to interpret user intentions, adapt to tone, and tailor responses dynamically (Wessel et al. 2025). Finally, the advanced reasoning property, reflecting GenAI's ability to simulate complex cognitive processes such as abstraction, inference, and synthesis, allows it to interpret ambiguous information, infer relationships, and generate logically consistent responses (Banh et al. 2025). Together, these properties enable new affordances in software development, expanding what developers can accomplish with GenAI.

As described by Pothukuchi et al. (2023), GenAI can potentially assist across all SDLC phases. For example, in the requirements analysis phase, GenAI and its conversational property may support the formulation of business requirements. In the design phase, its variability property may help suggest potential software architectures and identify weaknesses in designs. During development, GenAI and its advanced reasoning property may offer code completion, and in testing, it may facilitate automated testing and code understanding. Finally, in deployment and maintenance, GenAI may support software installation, performance monitoring, and code adaptation.

Several studies have examined the impact of GenAI assistance on software developers' work in controlled experimental settings, demonstrating productivity gains (Imai 2022; Peng et al. 2023; Ulfsnes et al. 2024; Wang et al. 2024a) that are strongest for simple, repetitive coding tasks (Ulfsnes et al. 2024). While developers using GenAI become more productive, GenAI-generated code often lacks the quality of human pair programming (Imai 2022). Its effectiveness diminishes when applied to complex tasks that require conceptual modeling and creativity, rather than merely producing syntax-compliant code components (Wang et al. 2024a). Additionally, GenAI improves code comprehension, offering better support than traditional web

resources for both students and professionals (Nam et al. 2024).

Previous research on GenAI in software development has primarily focused on experimental evidence of specific technical effects, particularly productivity gains. While these studies provide valuable insights, they do not provide a holistic understanding of the broader sociotechnical implications of GenAI adoption. Accordingly, we next introduce the sociotechnical perspective in information systems as the theoretical lens for our analysis.

## 2.2 Sociotechnical Perspective in Information Systems

The sociotechnical perspective is a foundational concept in IS research. It emphasizes the interplay between the technical and social components within work systems, in our context IT organizations, which are mutually dependent and interact to achieve organizational goals (Ryan et al. 2002; Bostrom et al. 2009; Sarker et al. 2019). The technical component refers to the IT artifacts used in an organization, including the hardware, software, and techniques necessary to execute work (Chatterjee et al. 2021). IT artifacts cannot create value in isolation, but must be embedded in the social structures of organizations. Thus, the social component refers to an organization's individuals, teams, interpersonal relationships, social capital, knowledge, skills – in short, the human factor (Ryan et al. 2002).

The technical and social components interact to achieve both instrumental and humanistic objectives. Instrumental objectives are directly linked to measurable business outcomes, including productivity improvements in workflows and economic gains such as increased profitability (Sarker et al. 2019). Humanistic objectives, in contrast, focus on the individuals who interact with and are affected by IT, emphasizing their emotions, concerns, professional identities, and overall experience with the technical component (Mumford 2006; Sarker et al. 2019; Melville et al. 2023).

We build upon prior IS research that adopts the sociotechnical perspective to examine the organizational impacts of IT adoption across various contexts (Ryan et al. 2002). In line with this approach, we provide theoretical background on three key aspects of the social component of sociotechnical systems: social capital in organizations, machine-induced learning and reflection, and professional identity in IT. These aspects are subject to academic discourse in the IS field and likely both influence and are influenced by the adoption of GenAI within IT organizations, which we explore further in our analyses.

## 2.3 Social Capital in Organizations

Social capital is an essential aspect of the social component of sociotechnical systems (Sarker et al. 2019). Social capital captures the idea that interconnected individuals can access tangible and intangible resources through their personal and professional networks (Adler and Kwon 2002). For software developers, this includes drawing on the expertise, time, and mental capacity of colleagues to address work-related challenges or support personal development. Prior research indicates that social capital enhances organizational outcomes by improving team cohesion, collaboration, and, thereby, increasing productivity (Reagans and Zuckerman 2001; Yang et al. 2009; Clopton 2011), but also benefits individuals in their career success (Seibert et al. 2001).

To accumulate social capital within organizations, a key concept is “organizational citizenship behaviors” (Organ 1988). Organizational citizenship behaviors describe employees supporting one another in daily interactions, particularly in areas beyond formal role requirements and outside explicit recognition in the reward system (Bolino et al. 2002). In the context of software development, this can involve colleagues sharing knowledge about coding practices and debugging strategies. This fosters frequent communication among colleagues, which is essential for building interpersonal relationships (Szulanski 1996; Joshi et al. 2007).

The rise of GenAI in software development may influence social capital in several ways. On the one hand, GenAI could be a pitfall for social capital accumulation by acting as a substitute for organizational citizenship behaviors. If developers viewed GenAI systems as digital partners instead of traditional tools, they could reduce mutual support and communication among their human colleagues. On the other hand, GenAI offers functionalities that may even improve the quality of collaborations and knowledge exchange, such as real-time language translation, addressing the need for supportive technologies previously emphasized in the literature on social capital (Huysman and Wulf 2006). In either case, by examining the role of GenAI in shaping social capital within software development, we contribute to IS research aiming to understand the antecedents and importance of social capital both broadly (e.g., Weiler et al. 2022) and within software development specifically (e.g., Yang et al. 2009; Xiang et al. 2013).

## 2.4 Machine-Induced Learning and Reflection

The rise of intelligent machines has sparked a recent stream of IS research examining how they enable humans to learn and engage in self-reflection. Empirical evidence shows

that interacting with intelligent machines to solve a task can lead individuals to adjust their mental models, that is, change their understanding of the world (e.g., Bauer et al. 2023). Similarly, such interactions can prompt self-reflection on one's knowledge and assumptions (Jussupow et al. 2021; Pfeuffer et al. 2023). These existing studies, however, mostly focus on machines that are based on "traditional" AI for classification and regression tasks.

GenAI-powered machines provide new opportunities for learning and reflection. Unlike traditional AI, which focuses on classification and regression tasks, GenAI can generate content in response to user prompts, as seen in large language model-based chatbots. Unsurprisingly, prior research repeatedly highlights such GenAI-based systems as a substantial opportunity for enhancing human learning (Yan et al. 2024). This is especially relevant in software development, where GenAI can, for example, support machine-induced learning by mapping programming code to natural language, thereby helping students understand and develop problem-solving strategies (Suh and An 2022). However, GenAI also presents pitfalls: novice developers may still struggle with the same difficulties but get an "illusion of competence", falsely believing they understand more than they actually do (Prather et al. 2024). By examining developers' perspectives on GenAI as a teacher and coding partner, we contribute to the theory of machine-induced learning and reflection: Our study shows if and how prior findings from this emerging body of research may apply to GenAI-based systems.

## 2.5 Professional Identity in IT

Another important aspect of the human factor in sociotechnical systems is the professional identity of practitioners. Professional identity refers to the extent to which individuals define themselves as members of a profession and the value they attach to this identification (Riemenschneider and Armstrong 2021). Indeed, individuals often develop strong attachments to their work, aligning themselves with the goals, values, norms, and practices associated with their profession (Pratt et al. 2006; Stein et al. 2013). In the IT sector, professional identity is directly linked to job satisfaction and affective commitment (Brooks et al. 2015), which, in turn, influence employee retention and overall organizational performance (Riemenschneider and Armstrong 2021).

More recently, researchers have examined the role of AI in shaping employees' professional identity in the workplace (e.g., Strich et al. 2021; Jussupow et al. 2022; Mirbabaie et al. 2022). The introduction of AI in organizations may present a pitfall in the sense that it can undermine employees' identification with their profession due to fears of job replacement (Mirbabaie et al. 2022). Moreover,

employees may perceive AI as a threat to multiple dimensions of their professional identity, including their status position, professional autonomy, and their perceived relevance as experts (Jussupow et al. 2018). Notably, prior research indicates that when facing such identity threats from AI, employees may adopt various strategies to protect and reinforce their professional identity, such as demonstrating expertise beyond AI's capabilities or resisting the adoption of AI within their organizations (Craig et al. 2019; Strich et al. 2021; Jussupow et al. 2022).

The shift from delegation to co-creation between developers and GenAI-driven IT artifacts may have significant implications for professional identity and status. This study contributes to the literature by examining these potential shifts in developers' professional identity in software development work. Specifically, we explore how software developers perceive and respond to GenAI, and how these perceptions and reactions align with or challenge existing theoretical frameworks on professional identity.

## 3 Methodology

### 3.1 Research Design

Our goal is to examine the opportunities and pitfalls of GenAI in software development through a sociotechnical lens (Sarker et al. 2019). This perspective remains under-explored in IS research, making it all the more important to examine in order to fully understand both the technical and social components of GenAI adoption and how they interact. At the same time, applying this lens is challenging, as it requires both a broad perspective and a context-sensitive research setting. Such a setting must allow for insights into instrumental outcomes, such as the direct and indirect productivity effects of GenAI adoption, as well as humanistic outcomes, such as changes in software developers' professional identity and well-being. To address these requirements, we employ a qualitative research design, specifically, gathering data through interviews and analyzing the data based on grounded theory to develop a conceptual model (Gioia et al. 2013).

Qualitative research is well-suited and widely established for investigating emerging research areas in IS (Gioia et al. 2013). It enables the interpretation of findings within the organizational settings where the phenomena of interest occur (Strauss and Corbin 1998; Gioia et al. 2013). Such research is typically characterized along two dimensions: (1) data-centric vs. interpretation-centric and (2) inductive vs. deductive (Sarker et al. 2018). In line with this categorization, we adopt a data-centric, inductive approach. We collect qualitative data through semi-structured interviews with software development practitioners

from multiple organizations, viewing these practitioners as knowledgeable informants and leveraging their perspectives to uncover new insights and construct a generalizable conceptual model. This approach aligns with our research goals, as semi-structured interviews are particularly suited for investigating under-researched topics and gathering information that might otherwise be difficult to obtain (Myers and Newman 2007). It is important to note that the findings reflect subjective impressions from developers and do not necessarily reveal causal mechanisms in the way that, for example, a randomized field experiment would. Our aim, rather, is to help identify causal relationships that emerge from the data. Thus, following the guidance of Klein and Myers (1999), we formulate empirically grounded theoretical propositions within a conceptual model.

### 3.2 Data Collection

We selected our interviewees through purposive sampling (Etikan et al. 2016): Our main criterion for interview partners was solid expertise and practical experience of at least one year in software development within a professional setting; consequently, we excluded amateur developers, students lacking practical experience, and persons with similar profiles. Guided by the principle of a variety of voices from Myers and Newman (2007), we did not constrain our search to experts currently engaged in a software development position, but deliberately included experts in related positions. These roles include technology consultants, product owners, business analysts, or different management roles, provided they have a software development background as previously defined. Interviewing experts with different job positions allows us to investigate the topic from multiple angles (Myers and Newman 2007), providing us with a more comprehensive picture of how GenAI systems are perceived in the software industry. Table 1 presents a detailed list of our interview partners. Our interviewees come from 13 companies from different sectors and specializations, including tech consultancies, E-commerce companies, and IT service providers. These companies also vary in their size, ranging from startups with fewer than 50 employees to large enterprises exceeding 100,000 employees. We recruited our experts through emails, social media channels (primarily LinkedIn), and snowball sampling from earlier participants (Negrin et al. 2022).

We started all interviews with a short description of the research project and a self-introduction by the interviewee. To guide our semi-structured interviews, we developed a set of questions to elicit practitioners' views on multiple aspects of GenAI, such as experienced individual and organizational productivity effects, challenges, and perceived changes in their professional identity. At the end of

each interview, we gave interview partners time for further remarks to ensure the collection of information that we did not elicit directly. We developed the questionnaire in multiple iterations within our research group and conducted one pretest interview before finalizing our questionnaire. We provide an overview of our interview questions in the online appendix. We organized each interview according to these guiding questions, but encouraged practitioners to speak freely and, occasionally, deviate from the guideline. More specifically, we allowed new themes, not initially covered by our questionnaire, to emerge during the interviews whenever experts regarded them as relevant in specific contexts.

The interviews, ranging from 45 to 60 min, took place from November 2023 to July 2025, using Zoom or Microsoft Teams. With the participants' permission, we recorded these sessions and transcribed them to enable qualitative content analysis.

### 3.3 Data Analysis

The goal of our analysis is to extract meaningful empirical insights from our qualitative data and develop a coherent, generalizable conceptual model that captures the broad impacts of GenAI on the software development discipline. To accomplish this, we build upon the Gioia methodology for inductive research, which provides a structured approach to qualitative data analysis that is widely used in the IS literature (Gioia et al. 2013). This methodology is particularly suited for exploratory research with limited prior theory, allowing insights to emerge directly from the collected data rather than being affected by preconceived concepts and ideas (Gioia et al. 2013; Sarker et al. 2018).

The qualitative content analysis by Gioia et al. (2013) follows a three-step procedure. In the first-order analysis, researchers engage in open coding, identifying meaningful informant terms – or first-order concepts – within the raw data without any further interpretation or abstraction. In the second-order analysis, researchers look for similarities among the first-order concepts and organize them into more abstract clusters termed second-order concepts. Finally, if the underlying data allows it, researchers group the second-order concepts into a higher abstraction level, the aggregate dimensions. Based on these first-order and second-order concepts and the aggregate dimensions, the researchers construct the so-called data structure, which serves as an abstract representation of the qualitative data.

As recommended by O'Connor and Joffe (2020), we engaged two researchers in the full coding process independently using a subset of the transcript data to reduce personal biases and subjective interpretations. Initially, the two researchers familiarized themselves with the raw data and applied the first step of the coding process, which is the

**Table 1** Details of interviewed practitioners

| Interviewee ID | Org. | # of employees | Location               | Sector                                 | Job position              | Years of exp. |
|----------------|------|----------------|------------------------|----------------------------------------|---------------------------|---------------|
| I1             | A    | 1000+          | Europe (international) | Industr. digital transformation        | Software developer        | 11            |
| I2             | A    | 1000+          | Europe (international) | Industr. digital transformation        | Tech consultant           | 5             |
| I3             | A    | 1000+          | Europe (international) | Industr. digital transformation        | Software developer        | 10            |
| I4             | A    | 1000+          | Europe (international) | Industr. digital transformation        | Machine learning engineer | 2             |
| I5             | A    | 1000+          | Europe (international) | Industr. digital transformation        | Software developer        | 5             |
| I6             | A    | 1000+          | Europe (international) | Industr. digital transformation        | Software developer        | 4.5           |
| I7             | B    | 13.000+        | Germany                | IT consulting                          | Senior manager            | 20            |
| I8             | B    | 13.000+        | Germany                | IT consulting                          | Software engineer         | 2             |
| I9             | B    | 13.000+        | Germany                | IT consulting                          | Business analyst          | 1.5           |
| I10            | B    | 13.000+        | Germany                | IT consulting                          | Software developer        | 8.5           |
| I11            | B    | 13.000+        | Germany                | IT consulting                          | Software engineer         | 1.5           |
| I12            | C    | 500+           | Germany                | ERP software development               | Quality manager (Testing) | 35            |
| I13            | D    | 250+           | Germany                | E-commerce                             | Product owner             | 2.5           |
| I14            | E    | 300+           | Germany                | ERP software development               | Software architect        | 20            |
| I15            | D    | 250+           | Germany                | E-commerce                             | Team leader backend progr | 7             |
| I16            | F    | 40.000+        | Europe (international) | IT service provider                    | Software developer        | 20            |
| I17            | C    | 500+           | Germany                | ERP software development               | Software developer        | 20            |
| I18            | G    | 300.000+       | Global                 | Industry group                         | Software developer        | 2.5           |
| I19            | C    | 500+           | Germany                | ERP software development               | Software developer        | 27            |
| I20            | H    | 200+           | Germany                | ERP software development               | Software developer        | 20            |
| I21            | I    | 4000+          | Europe (international) | ERP software development               | Quality manager           | 10            |
| I22            | J    | <50            | Europe (international) | Financial Technology (FinTech)         | Software engineer         | 8             |
| I23            | K    | 100.000+       | Global                 | ERP software development               | Team leader AI            | 7             |
| I24            | J    | <50            | Europe (international) | Financial Technology (FinTech)         | Data scientist            | 7             |
| I25            | J    | <50            | Europe (international) | Financial Technology (FinTech)         | Software developer        | 7.5           |
| I26            | J    | <50            | Europe (international) | Financial Technology (FinTech)         | Data scientist            | 5             |
| I27            | J    | <50            | Europe (international) | Financial Technology (FinTech)         | Software engineer         | 7             |
| I28            | L    | 8000+          | Europe (international) | IT service provider accounting and tax | Data scientist            | 19            |
| I29            | L    | 8000+          | Europe (international) | IT service provider accounting and tax | Requirements engineer     | 1.5           |
| I30            | M    | 500+           | Germany                | Travel technology company              | Frontend engineer         | 5             |

identification of first-order concepts without further abstraction. The first coder identified 14 first-order

concepts containing 115 codes, whereas the second coder identified 21 first-order concepts containing 77 codes. With

the help of a third researcher as a mediator, the two coders compared their code assignments and discussed any discrepancies. One example of an emerging discrepancy pertains to the quotes *The review by colleagues, human, flesh-and-blood colleagues, helps to create a kind of connection to the colleague and thus to understand each other better on some level, which promotes collaboration, even if it really is just small, trivial errors. and I would assume that the exchange of knowledge is more likely to be inhibited because I can also discuss problems with my virtual co-pilot [...]*. Whereas coder 1 assigned both passages equally to a first-order concept labeled *collaboration between humans*, coder 2 assigned the first passage to a concept labeled *Negative effect on interpersonal relationships*, and the second passage to a concept labeled *Reduction of communication within software development teams*, acknowledging the difference between personal and functional dimensions of team interactions. After discussing the issue with the mediator, the researchers resolved the discrepancy by agreeing on the more detailed classification of coder 2.

Afterwards, the two coders revisited the raw data in an iterative process, during which additional codes and first-order concepts emerged, with each discrepancy discussed and resolved similarly. In later iterations, both coders independently conducted second-order analyses and the creation of the aggregate dimensions. In the later stages of the data analysis, the full research group engaged in several discussions on the final refinement of the data structure. For instance, we revisited the aforementioned discussion about the concepts related to the impact on human collaboration. We concluded that the increasingly independent working practices of developers are highly interrelated with the less frequent human interactions, with the latter being a direct consequence of the former. Thus, we finally defined the code *increased developer independence and less frequent human interactions*. Another example worth reporting concerns the upcoming skill requirements for future developers. Initially, we grouped all related observations under a single broad code capturing general skill demands. However, in later discussions, we recognized two distinct dimensions of skill requirements in the data. As a result, we split the initial code into two separate first-order concepts: *Shifting skill requirements: GenAI-usage skills*, reflecting the technical proficiency required to effectively interact with GenAI systems, and *Broad conceptual and analytical skillset*, which reflects the growing need for developers to think in terms of systems, architectures, and problem-solving skills that become more important as their primary role shifts from writing code to overseeing and integrating AI-generated outputs into software systems. Finally, one late adjustment of our data structure concerned the concept of opportunities and pitfalls for individual software

developers. We initially used the second-order concept *individual performance*, but later recognized the need to distinguish between two distinct forms: effects on developers' *individual task performance*, and effects on their *personal development and well-being*.

Finally, based on the agreed-upon preliminary data structure, one researcher conducted the coding of the entire transcripts (O'Connor and Joffe 2020). To achieve theoretical saturation (Gioia et al. 2013; Glaser and Strauss 2017), the researcher remains open to new concepts that could emerge from the rest of the data, discussing any adjustments to the data structure with the full research team. During this final coding stage, we revised the data structure multiple times until we converged to a final version that all researchers agreed upon.

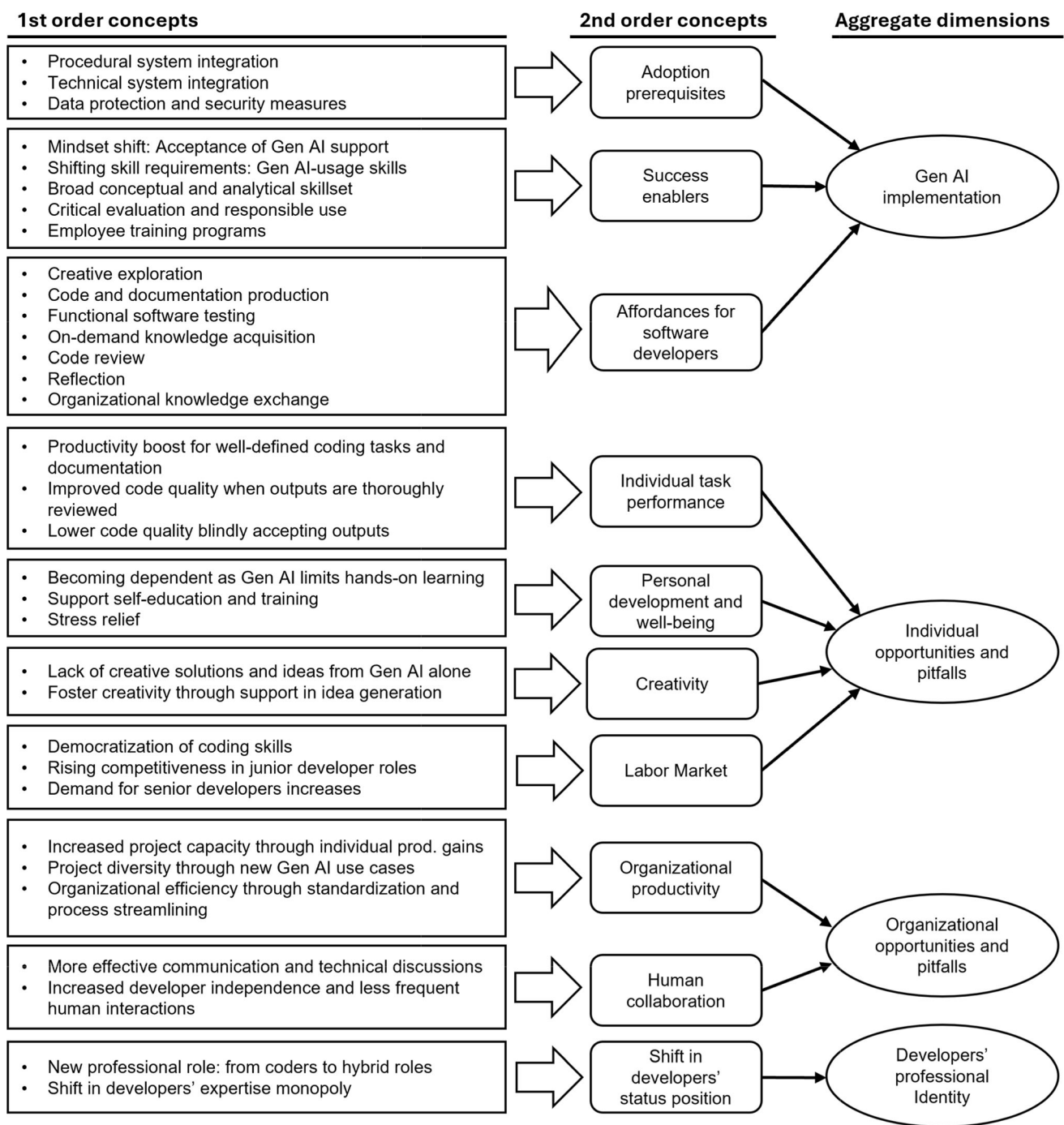
The data structure represents an abstract representation of our qualitative data, or, to put it in the words of Gioia et al. (2013), a "static picture of a dynamic phenomenon". As the final step in our grounded theory approach, we connect the concepts of the data structure and integrate them into existing theory, thereby constructing a conceptual model that holistically captures the drivers and effects of GenAI on software developers and organizations.

## 4 Results

We now detail the identified opportunities and pitfalls of GenAI for software developers and IT organizations, presenting the underlying data structure in Fig. 1 and the resulting conceptual model in Fig. 2. While the data structure follows the Gioia methodology and is fully grounded in our qualitative data, the conceptual model also incorporates theoretical insights from existing literature and connects them with our empirical findings. We provide further details on our first-order concepts, second-order concepts, and aggregate dimensions, including exemplary quotes from experts, in Tables 4 to 8 in the online appendix. In the following subsections, we organize the details of our findings in line with the conceptual model.

### 4.1 Overview of the Conceptual Model

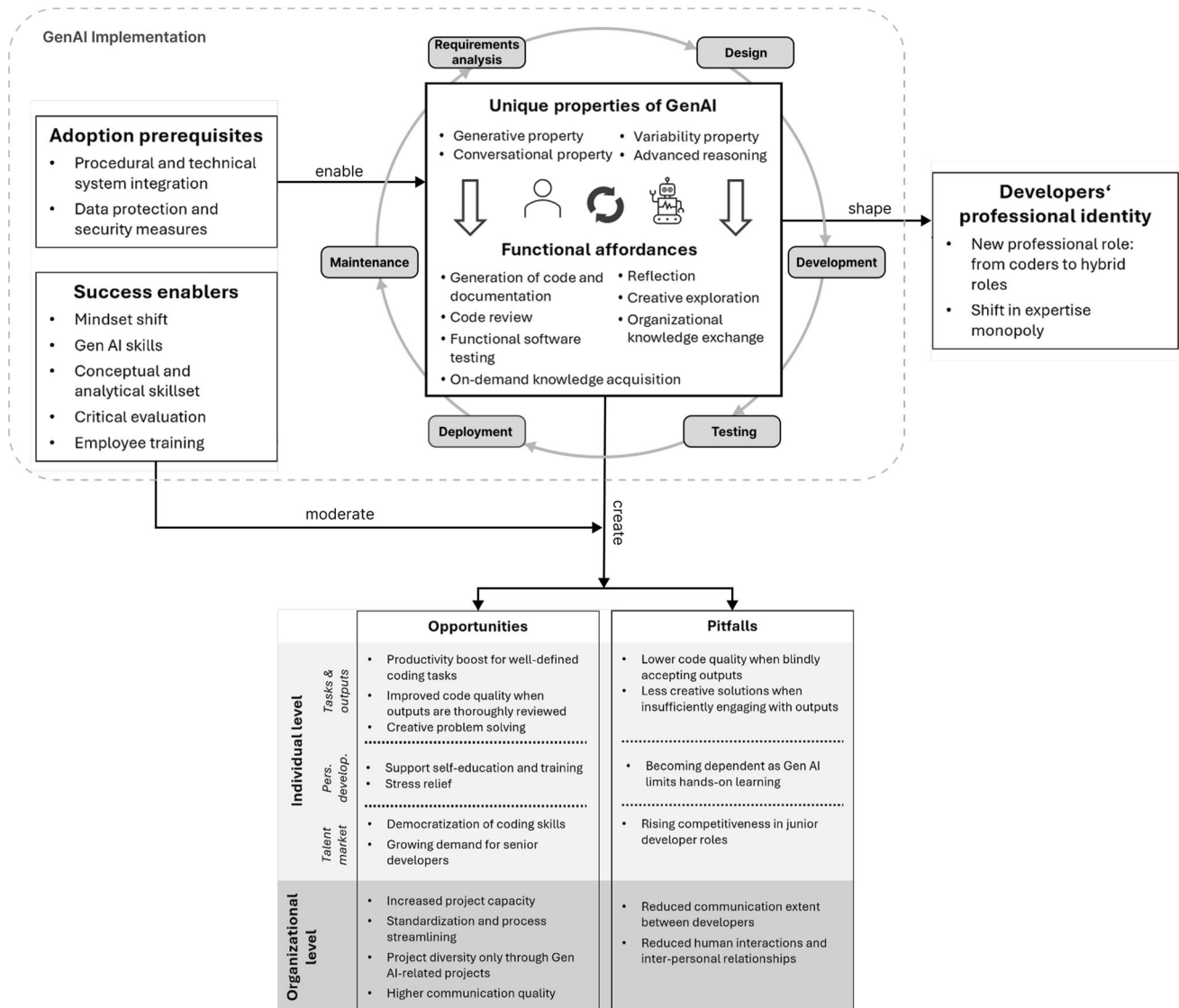
Our conceptual model aims to capture the sociotechnical impact of GenAI on individual software developers and the organizations they are part of. It contains four overarching dimensions: GenAI implementation, individual opportunities and pitfalls, organizational opportunities and pitfalls, and the software developers' professional identity. The GenAI implementation dimension refers to the adoption prerequisites that management must establish and the success factors at the team and individual levels that enable the effective adoption of GenAI systems in IT



**Fig. 1** Resulting data structure following Gioia et al. (2013)

organizations. A set of unique properties makes GenAI systems distinct and enables their use by developers through specific functional affordances (Markus and Silver 2008). Developers apply these functional affordances throughout the SDLC, although not all phases are affected to the same extent. Applying these functional affordances can have various effects on the sociotechnical systems of IT organizations and, thereby, create opportunities and

pitfalls. On the individual level, these affect developers' tasks, personal development, and talent market prospects. On the organizational level, opportunities and pitfalls include operational productivity and team collaboration. The extent to which opportunities outweigh pitfalls is shaped by the success enablers that developers and organizations employ to moderate the effects of GenAI implementation. Finally, the software developers'



**Fig. 2** Conceptual model of the opportunities and pitfalls of GenAI for software developers and organizations

professional identity dimension describes how GenAI impacts the perception and role of software developers not only within their organizations but in society in general. In the following sections, we explain each aggregate dimension in detail.

## 4.2 GenAI Implementation

### 4.2.1 Adoption Prerequisites

IT organizations aiming to implement GenAI systems into their operations need to fulfill crucial implementation requirements. From a procedural perspective, the management must establish clear rules and boundaries regarding the use of GenAI systems (I4, Machine learning engineer). These rules should not only specify what tasks

GenAI systems are permitted to perform but also define the processes where their involvement is desired and where it is not (I12, Quality manager (Testing); I20, Software developer). For instance, the management could specify that GenAI systems are permitted to assist with simple, repetitive coding tasks whereas critical tasks, such as the architecture design of a software product, should be performed by senior developers alone. This requirement is best illustrated by the following quote:

*“Clear rules need to be defined by the management in order to prevent misuse. You simply have to say: How can it be used and what must not be used”* (I12, Quality manager (Testing)).

From a technical perspective, organizations must ensure that GenAI systems are interoperable with the existing system and tool landscape, avoiding conflicts with other

tools. Seamless integration prevents disruptions and allows for frictionless usage, ensuring that GenAI systems do not interrupt software development workflows (I3, Software developer). Insufficient technical integration can severely hinder effective utilization of these systems. For example, I29 (Requirements engineer) describes a situation where his organization used multiple different GenAI systems, one provided as a commercial enterprise GenAI service and another operated internally as a proprietary instance, with access unevenly distributed across employees due to licensing limitations. As a result, custom GPTs developed for specific departments could not be shared with, e.g., service teams, who lacked the necessary access. This fragmentation of tools prevents knowledge sharing and creates collaboration barriers within the organization.

Lastly, organizations must implement data protection and general security measures to avoid reputational and legal problems (I6, Software developer). Most importantly, the management must provide conditions that prevent sensitive information from being inadvertently disclosed to third parties. Possible security measures include hosting GenAI models on in-house servers (I2, Tech consultant) and enforcing strict policies, such as prohibiting the input of critical proprietary code to prevent unintended data exposure (I17, Software developer).

#### 4.2.2 Affordances for Software Developers

The aforementioned requirements enable developers to use GenAI systems seamlessly and securely through several functional affordances (Markus and Silver 2008). Grounded in our qualitative data, we identify seven different functional affordances: generation of code and documentation, code review, functional software testing, on-demand knowledge acquisition, reflection, creative exploration, and organizational knowledge exchange.

The *generation of code and documentation* affordance allows developers to generate first drafts of code, documents, and other artifacts that can be further adapted and refined if needed. Unlike traditional coding assistants that mainly provide autocomplete suggestions or static templates, GenAI can create entire code segments, structured comments, and explanatory texts tailored to specific project contexts. This transformation is enabled by several unique properties of GenAI, more specifically, its generative, variability, and conversational properties (Feuerriegel et al. 2024; Storey et al. 2025). The generative property enables the creation of complete solutions that do not yet exist in any pre-defined form. The conversational property allows developers to shape these solutions by providing precise requirements and iteratively refining outputs through natural-language interaction. If a solution does not meet their

needs, the variability property enables developers to re-run an initial prompt and obtain alternative solutions.

The *code review* affordance includes all activities where developers assess the quality of source code with regard to its readability, structure, and potential errors. While traditional software tools have supported code review by flagging potential issues and offering predefined auto-fixes, GenAI enhances this process by providing explanatory feedback and generating context-aware improvement suggestions. This is mostly enabled by the advanced reasoning and generative properties of GenAI (Banh et al. 2025; Wessel et al. 2025). The advanced reasoning property enables the identification of structural and logical connections across the code, thereby highlighting even subtle issues while simultaneously providing explanations that mimic human reasoning. Additionally, the generative property allows developers to obtain the improved code that directly addresses the identified issues.

The *functional software testing* affordance includes evaluation of whether software units behave as expected, e.g., via unit tests, rather than focusing on the code quality. Unlike traditional tools for testing such as linters, compilers, or static analyzers that apply fixed rules, GenAI provides context-aware feedback, detects logical inconsistencies, proposes tailored test cases, and even generates potential fixes with natural-language explanations. Similar to code review, functional software testing is primarily enabled by the advanced reasoning and generative properties of GenAI (Banh et al. 2025; Feuerriegel et al. 2024). Advanced reasoning enables the interpretation of a software unit's intended behavior and execution logic as well as the detection of inconsistencies. The generative property enables the production of tailored test cases, synthetic test data, and potential fixes.

The *on-demand knowledge acquisition* affordance involves using GenAI systems to rapidly gain knowledge relevant to development tasks, such as background knowledge about certain software libraries. GenAI significantly enhances developers' knowledge acquisition by providing context-sensitive explanations, examples, and comparisons tailored to the developer's immediate task, in contrast to traditional tools that require searching static documentation or online forums for closely related information. This shift is facilitated by the advanced reasoning and conversational properties of GenAI (Banh et al. 2025; Chang et al. 2024). The advanced reasoning property enables the synthesis of relevant information and the generation of explanations and contextually meaningful examples. The conversational property, in turn, allows developers to engage in natural-language dialogue and ask follow-up or clarifying questions, thereby supporting deeper exploration.

The *reflection* affordance involves developers using GenAI to obtain a critical second opinion on their current work progress. Similar to on-demand knowledge acquisition, reflection is largely enabled by the conversational and advanced reasoning properties of GenAI (Banh et al. 2025; Ramaul et al. 2024). The conversational property allows developers to use GenAI as a sparring partner, iteratively asking for feedback and receiving alternative suggestions, thereby progressively refining their work. The advanced reasoning property enables the evaluation of the coherence of developers' solutions and their reasoning, the identification of potential weaknesses or inconsistencies, and the creation of alternative, contextually aware proposals.

The *creative exploration* affordance refers to any open brainstorming activity where developers interact with GenAI to come up with new, creative ideas. This affordance is mostly facilitated by generative, variability, and conversational properties of GenAI (Banh and Strobel 2023; Storey et al. 2025). The generative property enables developers to generate diverse ideas, images, and prototypes. The variability property extends this exploratory space by allowing developers to regenerate outputs and examine potentially alternative suggestions for the same input. Additionally, the conversational property enables the interaction between developers and GenAI systems, facilitating brainstorming through iterative idea-feedback exchanges.

Lastly, the *organizational knowledge exchange* affordance involves developers leveraging GenAI to retrieve, synthesize, and share knowledge from disparate internal sources (e.g., documentation, chats, or code repositories), enabling unified access to organization-specific information. Unlike traditional tools, which require developers to manually search, compare, and integrate documentation, tickets, or code comments, GenAI can synthesize disparate inputs into a unified overview. This transformation is facilitated primarily by the advanced reasoning property of GenAI (Banh et al. 2025). Advanced reasoning enables the semantic integration of disparate organizational knowledge by identifying relationships across fragments, resolving inconsistencies, and producing coherent summaries.

#### 4.2.3 Success Enablers

These functional affordances have a strong impact on the practices of software developers and the overall operation of their teams and organizations. This impact can take the form of opportunities, such as improved workflows and positive economic outcomes for organizations, but also of pitfalls, meaning unforeseen challenges and risks that may arise from the integration of GenAI into development processes. Notably, one recurrent theme that we encounter in our qualitative data is that the software developers

themselves determine whether the effects are positive or negative. In particular, we identify five success enablers that moderate the overarching effects of GenAI interactions: A general mindset shift among developers is necessary, requiring them to accept that certain aspects of their work can be either fully accomplished or significantly enhanced by GenAI (I2, Tech consultant). This goes hand in hand with a reassessment of human strengths and limitations; developers should recognize the tasks in which GenAI is superior and be willing to delegate those accordingly. This idea is illustrated by the following quote:

*“Developers are sometimes a little reluctant to use AI because they say, yes, it’s bad. What comes out of it is rubbish, and we don’t need that. At least, that’s very often the case in our company. And I think that sometimes developers need to rethink their attitude and realize that AI can really help them, that it can perhaps also lead to a rethink in the way they work”* (I29, Requirements engineer).

Another crucial success factor and upcoming “key competence” (I8, Software engineer) is the skill to effectively use GenAI systems. Most importantly, developers must understand how to write prompts in a way that steers the machine towards the desired goal (e.g., I7, Senior manager), a skill typically referred to as “prompt engineering” (I26, Data scientist). At the same time, the traditional division of labor, where senior developers design the software architecture and junior developers write the source code, is about to disappear. With faster development cycles, all developers are now expected to actively engage in architectural, security, and domain-related aspects (I23, Team leader AI). In addition to the broader technical expertise, there is a growing importance of soft skills: analytical reasoning, structured problem-solving, and creativity are becoming increasingly important (I16, Software developer; I20, Software developer). Furthermore, developers must be aware of potential errors and inconsistencies in generated code. For instance, I9 (Business analyst) mentions the example of inconsistencies with the programming guidelines of the proprietary enterprise resource planning (ERP) system. Developers must be able to detect and rectify such issues before pushing them into the code base.

Finally, to ensure that developers and non-technical staff can derive the most benefit from GenAI usage, organizations must invest in continuous training programs. These programs should equip employees with a clear understanding of GenAI functionalities, best practices for prompt formulations, and awareness of its limitations (I11, Software engineer).

### 4.3 Opportunities and Pitfalls for Individual Software Developers

#### 4.3.1 Individual Task Performance

If these critical success factors are met, the functional affordances provided by GenAI offer various opportunities for individual software developers. Confirming most experimental studies on productivity effects (e.g., Peng et al. 2023; Li et al. 2024; Ulfsnes et al. 2024; Wang et al. 2024a), our experts unanimously report strong productivity gains through GenAI usage in the development stage, which means that developers are empowered to produce considerably more output in a given time, be it software code, code documentation, or other miscellaneous work such as drafting technical reports and manuals. The resulting time savings allow developers to focus on more complex, conceptual, and ultimately more engaging work (e.g., I12, Quality manager (Testing); I23, Software engineer). A closer examination of these productivity gains reveals more nuanced insights. GenAI systems provide strong support for specific, well-defined code production tasks and can, in such cases, partially take over work typically done by human developers (I2, Tech consultant; I15, Team leader backend progr.). Importantly, GenAI support is especially helpful for tasks where developers do not have experience and an established routine, with routine being defined as regular engagement in these tasks and a learned standardized approach for solving them (Parnin et al. 2017; Masood et al. 2022). Therefore, junior developers benefit the most from GenAI interactions, as they often handle basic coding tasks and usually do not possess extensive long-term working routines (I8, Software engineer). However, for more abstract and complex tasks, the productivity gains may vanish, as formulating precise prompts that capture all necessary requirements can be both time-consuming and challenging (e.g., I3, Software developer).

Besides the individual productivity effects, GenAI support also impacts the overall quality and structure of code outputs. When applied correctly and responsibly, meaning that developers provide well-structured prompts, review generated code carefully, and refine outputs as needed (in other words, if aforementioned success factors are met), GenAI can indeed enhance overall code quality and cleanliness, making code more standardized and easier to maintain (I17, Software developer; I24, Data Scientist). Interviewees highlight that GenAI systems can catch small problems and inconsistencies, such as missing transformation steps or copy-paste errors and act as an initial reviewer to flag potential issues before they appear in testing. In contrast, if the success factors are not met and developers insufficiently engage with GenAI outputs, code

quality may decrease severely. Interviewees note that generated code can become overly complex and difficult to debug (I24, Data scientist). Code problems that initially remain undetected and spread throughout the code base are particularly risky for the overall code quality of software development projects (e.g., I19, Software developer; I29, Requirements engineer).

#### 4.3.2 Personal Development and Well-Being

GenAI does not only affect the speed and quality of software developer's work, it can also impact developer's professional development and well-being. One important opportunity lies in the support for self-directed learning. By engaging with code suggestions and actively trying to understand their underlying logic, software developers can learn new ways to approach complex programming problems (I1, Software developer; I26, Data scientist). In particular, self-directed learning of new programming languages becomes more efficient when supported by GenAI systems (I9, Business analyst; I11, Software engineer). However, excessive usage of these systems may also pose a risk to the skillset of developers in the long run. If developers constantly depend on generated outputs without engaging with the underlying concepts, they risk losing their expertise and practical skills, leading to increased reliance on these systems (e.g., I7, Senior manager; I14, Software architect). As I23 (Team leader AI) observes, the increased speed of code production, especially for junior colleagues, can bypass the slower, reflective learning process in which developers are forced to deeply think about problems and mistakes. Instead of critically evaluating why certain solutions work or fail, they may simply select from a set of suggested code options (I23, Team leader AI).

GenAI systems may also contribute to less stressful working conditions for software developers. The immense complexity, workload, and volatility of modern software development projects often lead to considerable stress for the involved employees (Windeler et al. 2017). We find that the certainty of having a digital partner as a "fallback plan" (I29, Data scientist) when a technical problem seems unsolvable reduces the feeling of being overwhelmed in overly complex situations. In particular, one crucial source of stress and unhappiness for software developers is the experience of becoming stuck on a problem, which disrupts the workflow (Graziotin et al. 2018) and creates a sense of being blocked (I23, Team Leader AI). In such moments, GenAI support can offer an initial direction toward a solution, helping developers regain their flow and, thus, reduce frustration (I22, Software engineer; I23, Team leader AI).

#### 4.3.3 Creativity

GenAI can also affect the way developers approach tasks that require creativity, particularly through the functional affordances of creative problem solving and reflection. A recurring theme is that GenAI itself does not have the ability to generate truly creative solutions. This limitation arises because GenAI models are trained on historical data and, therefore, lack the human creativity needed to create fundamentally new ideas. However, interactions with GenAI can still stimulate and enhance human creativity. This notion is captured by the following quote:

*"[...] This means that if my creativity has perhaps steered me in [...] a certain direction, then I have the opportunity to say: 'Dear AI, I'm at this point now. Maybe show me another way or other ways or am I right or am I wrong now?' This means that, in addition to your own creativity, you have the opportunity to broaden your horizon relatively easily and quickly and perhaps steer your own creativity in a completely different direction or to think completely differently [...]"* (I21, Quality manager).

Thus, GenAI may help developers achieve more creative solutions in complex problem-solving situations. By proposing multiple possible solution paths, GenAI facilitates ideation processes and helps developers come up with creative solutions. Thereby, GenAI provides thought-provoking impulses, even if the model itself would not have independently arrived at the complete solution (I2, Tech consultant; I4, Machine learning engineer; I29, Requirements engineer). Further, GenAI can act as a "sparring partner" (I21, Quality manager; I24, Data scientist) who constantly challenges developers' current solution paths. In this sense, GenAI does not act as a mere tool that supports developers execute their predefined coding tasks, but as a creative partner who proposes new, previously unconsidered solutions.

#### 4.3.4 Labor Market Effects

Further opportunities and pitfalls arise for software developers within the talent market. GenAI technologies lower traditional entry barriers, allowing individuals with limited formal training or experience to become productive in software development work more quickly (e.g., I2, Tech consultant; I3, Software developer). This can open up junior developer roles to a wider group of people (I1, Software developer) and contribute to the democratization of the field. Several interviewees highlight that even individuals with no coding background are now able to create working prototypes or automate simple tasks. However, this can lead to a "Dunning-Kruger effect" (I3, Software developer; I5, Software developer), where developers overestimate their actual abilities due to early successes

with GenAI-assisted software work. Relying too heavily on it may result in initial productivity but lead to stagnation when deeper problem-solving skills and technical knowledge are required (I2, Tech consultant).

While the reduction of entry barriers seems beneficial, it also leads to increased competition at the talent market. As GenAI streamlines and partially takes over many well-defined coding tasks typically assigned to junior developers, experts predict a reduced demand for junior roles in the next few years (e.g., I4, Machine learning engineer; I8, Software engineer). Put differently, organizations will become highly selective in their search for software development staff (I15, Team leader backend progr.), which will eventually "disrupt the lower-skilled software developers" (I8, Software engineer).

In parallel, the labor market for experienced senior developers is expected to remain strong, or even grow. The experts unanimously share the view that the demand for experienced developers, especially those who possess the skill to effectively interact with GenAI systems, will rise considerably. This is because organizations will need experienced developers who have a profound technical and conceptual understanding of the software products they build (I2, Tech consultant) and effectively coordinate and supervise the involved GenAI systems (I6, Software developer).

### 4.4 Opportunities and Pitfalls for IT Organizations

In addition to the previously discussed effects on the workflow and professional life of individual software developers, we now examine the opportunities and pitfalls that GenAI introduces at the organizational level.

#### 4.4.1 Organizational Productivity

Prior research has shown that although more advanced IT may be put in place and bring individual benefits, inefficiencies in processes, such as misallocation of resources or organizational "bottlenecks", may still limit organizational productivity, as seen in the "IT productivity paradox" (Brynjolfsson 1993). Still, we find that the integration of GenAI into organizational operations has the potential to enhance the overall productivity of IT organizations. This impact comes through multiple different channels. First, and most evidently, the productivity gains achieved at the individual level translate into greater overall work capacity. As developers complete specific tasks more quickly, organizations can take on a higher number of projects without proportionally increasing resources (*"In the past, a team could, for example, complete 3 projects in two areas within two years. Today, the same team might be able to complete 5 projects in three areas within two years."*) (I9,

Business analyst)). However, these productivity effects at the organizational level can only be realized if complementary functions, such as product management (I30, Frontend engineer), are able to keep pace with the increased development speed. While the number of feasible projects grows, the diversity of these projects grows only through those focused on implementing GenAI solutions within client organizations (I3, Software developer), such as the development of intelligent chatbots or other GenAI-based product features (I30, Frontend engineer). Another channel for organizational productivity increases is the potential to standardize and optimize organizational structures within software development workflows, leading to improved operational efficiency. The following expert quotation explains this notion:

*“Many processes in a company, especially in large organizations, are highly redundant—particularly in how architectures are designed and software is integrated into the overall infrastructure. Many developers end up reinventing the wheel. I see great potential in a centralized hub that consolidates this information and makes it available in a meaningful way”* (I16, Software developer).

There is a lively debate in the computer science literature about how software redundancies in complex software architectures are deliberately introduced to enhance error robustness (Carzaniga et al. 2009; Abouei Ardakan et al. 2016; Devi et al. 2023). Interviewee 16, however, refers to another kind of redundancy: the existence of multiple processes within an organization that address the same or similar problems (Armbrust et al. 2009). This type of redundancy causes inefficient usage of resources and reduced operational efficiency (Armbrust et al. 2009). GenAI has the potential to reduce such inefficiencies by acting as a central knowledge hub that consolidates information and prevents duplicate work. Several experts, for example, emphasized its ability to integrate content from fragmented “documentation pools” (I18, Software developer) or internal platforms like Confluence (I29, Requirements engineer), thereby supporting knowledge reuse and minimizing repeated documentation efforts. Beyond its function as central knowledge hub, GenAI may also streamline the planning and coordination of projects. By generating tickets based on project goals (I29, Requirements engineer), it can help structure roadmaps and promote standardization. When trained on tickets from earlier projects, GenAI can help minimize redundancies in planning by re-using standard tickets found in similar projects. Thus, as GenAI could automate certain coordination and management tasks, it holds the potential to transform traditional organizational structures by optimizing or reducing the need for some intermediary roles. As one interviewee emphasized, this could “increase the efficiency of information transfer from top to bottom” (I8, Software

engineer). Another hands-on example of how GenAI systems streamline project processes is the automatic generation of pull request summaries and explanations. This supports quicker code reviews and smoother team coordination, making project workflows more efficient (I23, Software developer).

#### 4.4.2 Human Collaboration

Beyond process improvements, GenAI also shapes the social component of IT organizations through its impact on team collaboration. Communication plays a crucial role in team dynamics as it fosters the emergence of interpersonal relationships (Szulanski 1996) and enables knowledge transfer (Joshi et al. 2007). Our experts report positive changes in the quality of communication, especially in team interactions. Team discussions become more dynamic as GenAI brings ideas when they are lacking, serving as a “basis for discussion” (I29, Requirements engineer) and supporting “knowledge exchange” (I8, Software engineer):

*“[...] and what we do a lot in the team is share these GPTs, and I don't think you can do that through normal communication, passing on all the knowledge they have to each individual. That means everyone in the team has the same knowledge that they can access, and you can then communicate on a different level, because I feel that everyone is more involved in the topic when you work with these GPTs. So I think it has significantly improved communication and collaboration within our team”* (I29, Requirements engineer).

Apart from the communication between software development peers, GenAI can also improve the quality of communication between technical and non-technical roles. It does so by making the transfer of explicit IT knowledge (Bassellier et al. 2001; Joshi et al. 2007), which often includes complex technical terminology, more comprehensible (I13, Product owner; I15, Team leader backend progr.). For instance, interviewee 13, a product owner of an online shop, highlights how GenAI helps articulate complex technical concepts to user experience teams, management, and other non-technical stakeholders. This aligns with the knowledge engineering paradox, which suggests that higher expertise levels make it harder to articulate and transfer knowledge (Liebowitz and Yan 2004). By simplifying explanations, GenAI mitigates this challenge, making technical insights more accessible.

Although we find that GenAI enhances communication quality within teams and serves as a valuable discussion aid, it can also result in an overall decrease in one-on-one interactions between developers. Developers increasingly use GenAI as a first point of contact for questions related to small, technical problems, replacing direct human interaction on basic programming issues (e.g., I10, Software

developer; I24, Data scientist). In particular, this shift is visible in interactions between junior and senior developers (I10, Software developer), as less experienced developers rely more on GenAI to help them solve problems faster when they are “stuck” (I25, Software developer; I27, Software engineer), without having to wait for a response from their senior colleagues. In this case, GenAI may cover “one cycle of improvements” (I22, Software engineer) or feedback, reducing “unnecessary communication” (I8, Software engineer).

However, frequent communication is a prerequisite for the emergence of interpersonal relationships between colleagues (Szulanski 1996). As one expert notes, even interactions around small, seemingly minor technical problems can provide valuable opportunities for building such relationships:

*“The review by colleagues, human, flesh-and-blood colleagues, helps to create a kind of connection to the colleague and thus to understand each other better on some level, which promotes collaboration, even if it really is just small, trivial errors. If I have these small, trivial errors corrected with the help of AI tools in the future, collaboration will of course also suffer”* (I2, Tech consultant).

This indicates that, although GenAI can strengthen communication quality, colleague-to-colleague interactions may inadvertently weaken interpersonal relationships and, eventually, team cohesion. This is a major pitfall for organizations as interpersonal relationships facilitate knowledge transfer and overall collaboration within teams (Szulanski 1996) and are vital for the loyalty of employees to the organization (Ammari et al. 2017).

#### 4.5 Developers' Professional Identity

GenAI also impacts the professional identity and self-image of software developers. As GenAI systems become increasingly embedded in software development workflows, experts expect a fundamental transformation of the professional role of software developers. Traditionally, software developers have strongly identified themselves as programmers (Marks and Scholarios 2007), with some even describing programming as a form of “art” (Jemielniak 2008). In contrast, over the next years, developers are expected to spend less time writing code directly, shifting their role toward new responsibilities (I16, Software developer). I15 (Team leader backend progr.) describes this transition as follows:

*“In seven years' time, I believe that we could be at a point where the pure writing of code no longer takes place as we know it today. In other words, as a developer, I will no longer have to sit down and write a class myself, for example, but it will be much more of a creative process:*

*You define contexts and more complex topics, but the writing of the actual source code can be done by artificial intelligence”* (I15, Team leader backend progr.).

This shift moves the professional role of software developers away from being software programmers towards a “hybrid” (I16, Software developer) role that emphasizes orchestrating various AI and non-AI tools. Instead of writing code line by line, developers will act as supervisors of GenAI systems, validating outputs, integrating them into larger systems, and ensuring they align with technical as well as business objectives.

The increasing automation of programming tasks also reshapes the expertise monopoly traditionally held by software developers. In the past, programming was viewed as a highly esteemed skill that required extensive training (I4, Machine learning engineer) and was possessed by only a few people. As the entry barriers to the job of software developers are lowered, this job is likely to be perceived as less challenging (I8, Software engineer) and, therefore, less valued, particularly by the public and non-technical colleagues (e.g., I5, Software developer; I13, Product owner). As a consequence, from the perspective of clients of software development firms, the demand for external developers' expertise and specialized knowledge may decline as clients start believing they can develop applications on their own with the help of GenAI (I7, Senior manager). The following quotes illustrate this devaluation:

*“What I've heard from clients is that since clients can write their application with LLMs themselves, software developers are being devalued”* (I7, Senior manager).

*“So it could easily be that there are people who then say, no one needs software developers, AI does it anyway”* (I5, Software developer).

#### 4.6 Linking Opportunities and Pitfalls to GenAI's Functional Affordances

In this subsection, we present how our data link GenAI's functional affordances to the observed effects, that is, to opportunities, pitfalls, and changes in developers' professional identity. Indeed, it is important to understand that the distinct affordances do not contribute equally to the realization of the aforementioned effects. Instead, as we present in Table 2, each effect is primarily driven by one or a specific subset of them. We derived the links between functional affordances and GenAI effects through a rigorous analysis of our qualitative data. For each quote associated with individual opportunities and pitfalls, organizational opportunities and pitfalls, and developers' professional identity, two researchers independently assigned the functional affordance(s) that participants indicated, either explicitly or implicitly, as the primary driver. Afterwards, the two researchers engaged in

**Table 2** Dominant affordances per SubEffect with illustrative quotes

| Affordance                            | GenAI effects (ordered by salience in our data)                            | Proposed affected SDLC phases                                    | Exemplary quotes                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| =Generation of code and documentation | Productivity boost for well-defined coding tasks and documentation         | Development                                                      | <i>Yes, code-related productivity is definitely increased. We can get suggestions from LLMs in a matter of seconds and use them in our work (I8, Software engineer).</i>                                                                                                                                                                                                                                                      |
|                                       | Becoming dependent as GenAI limits hands-on learning                       |                                                                  | <i>The coding level goes down. Right? The level of coding goes down because people don't have to solve everything from scratch anymore. Right? So that's a danger (I24, Data scientist).</i>                                                                                                                                                                                                                                  |
|                                       | New professional role: from coders to hybrid roles                         |                                                                  | <i>Back to the software developer. I don't think they will exist in this form in the future, because there are simply too many possibilities today. If I look five years into the future, so much code will be able to be generated automatically. Perhaps the term "software developer" won't even exist anymore and will be called something like AI engineer or something similar. (I16, Software developer).</i>          |
|                                       | Democratization of coding skills                                           |                                                                  | <i>If, for example, a tool makes programming easier, there is also the possibility that some tasks could be performed by people from other fields (I12, Quality manager (Testing)).</i>                                                                                                                                                                                                                                       |
| =Code review                          | Lower code quality when blindly accepting outputs                          | Development, Testing, Maintenance                                | <i>"[...] the code that's generated is so complex that it will have larger run times and becomes more complex to debug, I think [these] are definitely challenges, the first ones that I would think about." (I24, Data scientist)</i>                                                                                                                                                                                        |
|                                       | Increased developer independence and less collaboration                    |                                                                  | <i>I think the bigger effect is that you need to ask less questions to your peers because you have your own, as I said, sparring partner or copilot, whether that's integrated into your IDE or not. (I24, Data scientist).</i>                                                                                                                                                                                               |
| =Functional software testing          | Productivity boost for well-defined coding tasks and documentation         | Testing, Maintenance                                             | <i>Let's say I've written the code, but something isn't working. At first glance, it might be difficult to find the error in the code, such as a typo. It would be quicker to copy the code into ChatGPT and ask it to find the typo (I9, Business analyst).</i>                                                                                                                                                              |
|                                       | Organizational efficiency through standardization and process streamlining |                                                                  | <i>We basically built a custom GPT for [our product] and said, this is the current status, please create test cases for these scenarios. [...] We said that when a user logs in or selects an employee, the following should happen. Then he was able to derive structured test cases for us. And we were then able to automate these test cases with a ChatGPT Enterprise Custom GPT [...] (I29, Requirements engineer).</i> |
| =On-demand knowledge acquisition      | Support self-education and training                                        | Requirements analysis, Design, Development, Testing, Maintenance | <i>For highly skilled developers, the efficiency of learning a new programming language is significantly increased (I9, Business analyst).</i>                                                                                                                                                                                                                                                                                |
|                                       | Democratization of coding skills                                           |                                                                  | <i>On the other hand, software development itself is now more accessible than it used to be, which means that people who previously may have needed a lot of support to implement something are now asking ChatGPT, 'How can I easily automate this?' (I1, Software developer).</i>                                                                                                                                           |
|                                       | Increased developer independence and less collaboration                    |                                                                  | <i>We can be more independent because before, right, we were doing the Google search and I think AI is way more helpful in finding the information that you need. So it's more likely that if you need to do a research about something that you don't know, with Google search, it's likely that you don't find it what you need. But with AI, it's more likely that you find your answer. (I30, Frontend engineer).</i>     |

**Table 2** continued

| Affordance                        | GenAI effects (ordered by salience in our data)                            | Proposed affected SDLC phases                                    | Exemplary quotes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| =Reflection                       | Foster creativity through support in idea generation                       | Design, Development                                              | <i>That means, if I have [...] directed myself at a certain point in time or in a certain direction, then I still have the option of saying or asking: 'Dear AI, I am now at such and such a point, and so on and so forth. Perhaps show me another way or different ways, or am I now correct or am I now incorrect?'</i> (I21, Quality manager).                                                                                                                                                                                                                  |
|                                   | Increased developer independence and less collaboration                    |                                                                  | <i>Previously, if some junior developer coded something and he pushed his code, and then some senior developer is going to suggest, oh, this is not the right approach or improve this point over there. But now at their level, if we have integrated some generic tool to the code, they already have suggested the solution. At this point, you can improve your thing this or this is the best way or these four are the best ways. So you would cover one cycle of, you know, the improvement which was suggested by your senior</i> (I22, Software engineer). |
| =Creative exploration             | Foster creativity through support in idea generation                       | Design                                                           | <i>I think I'm more creative, at least from a different perspective, because you can say, yes, this is the problem, and it basically gives you possible solutions, and I think I would have come up with some of them myself if I had thought about it longer, but there are other things you might not have really thought of, and you think, oh yeah, that's crazy, and then you can think about it further, because it was just a little thought-provoking impulse from the AI with that little sentence or something.</i> (I29, Requirements engineer).         |
|                                   | Increased developer independence and less collaboration                    |                                                                  | <i>"I think the initial exchange has been replaced somewhat. Above all, junior developers today try to get the first idea for a solution with the help of LLMs. The first point of contact today would be LLMs"</i> (I10, Software developer).                                                                                                                                                                                                                                                                                                                      |
| Organizational knowledge exchange | Organizational efficiency through standardization and process streamlining | Requirements analysis, Design, Development, Testing, Maintenance | <i>What we do a lot in the team is share these GPTs, and I don't think you can do that through normal communication, passing on all the knowledge they have to each individual. That means everyone in the team then has the same knowledge, which they can access, and you can then communicate on a different level somehow</i> (I29).                                                                                                                                                                                                                            |

discussions on these affordance-effect assignments, compared and reconciled them until they converged toward the final linkage presented in Table 2. The GenAI effects are ordered by their salience in our data. Effects listed higher within an affordance were reported more frequently across interviews for that specific affordance. In the following, we briefly address several patterns that we find particularly noteworthy: the prominent role of the affordance to generate code and documentation, its potential risks for developers' workflows and professional growth, and the engagement-enhancing impact of affordances that stimulate cognitive involvement.

Several observed effects of GenAI on the software development domain stem from the affordance *generation of code and documentation*. The ability to rapidly generate

code and documentation allows developers to delegate large parts of the code production and, even for more complex architectures, start from a working version rather than from scratch. This affordance marks a fundamental shift in the workflows of developers who shift their focus from code production to coordinating GenAI systems, with multiple and wide-ranging effects on individual software developers. Besides the productivity-enhancing effects, however, this affordance can tempt developers to engage less in their programming work, which in turn entails multiple pitfalls, including eroding creativity and deskilling. Therefore, we must emphasize the crucial role of success enablers at this affordance in particular: Developers must not "succumb to the temptation" of blindly trusting the technology in programming tasks, but must use

it deliberately and responsibly while staying cognitively involved.

Not all functional affordances seem to entail pitfalls to the same extent. Especially those affordances that spark developers' cognitive processes (*on-demand knowledge acquisition*, *reflection*, and *creative exploration*) bring considerable opportunities for developers with comparably few pitfalls for their workflows and professional growth. Unlike code generation, these affordances mostly foster cognitive engagement. On-demand knowledge acquisition helps developers understand new concepts and learn new skills, reflection encourages careful re-evaluation of the status quo, and creative exploration supports creative problem solving. However, they may introduce a different kind of pitfall that affects the organizational level rather than the professional development of individual developers. On-demand knowledge acquisition, reflection, and creative exploration affect activities that were often performed in team settings (e.g., joint brainstorming with colleagues or mutual feedback sessions). Because these activities can be performed more independently with the help of GenAI, everyday peer interactions may decline, potentially weakening team cohesion and the collaborative culture in general.

For one affordance, we even exclusively find opportunities without any individual or organizational pitfall, that is, *organizational knowledge exchange*. Our data do not imply any drawback associated with using GenAI for retrieving, synthesizing, and sharing internal, organizational knowledge. Of course, pitfalls may still exist that were simply not represented in our data (e.g., biases introduced when GenAI synthesizes knowledge), but their absence in our findings remains noteworthy and highlights this affordance as a potential "low-hanging fruit" for organizations.

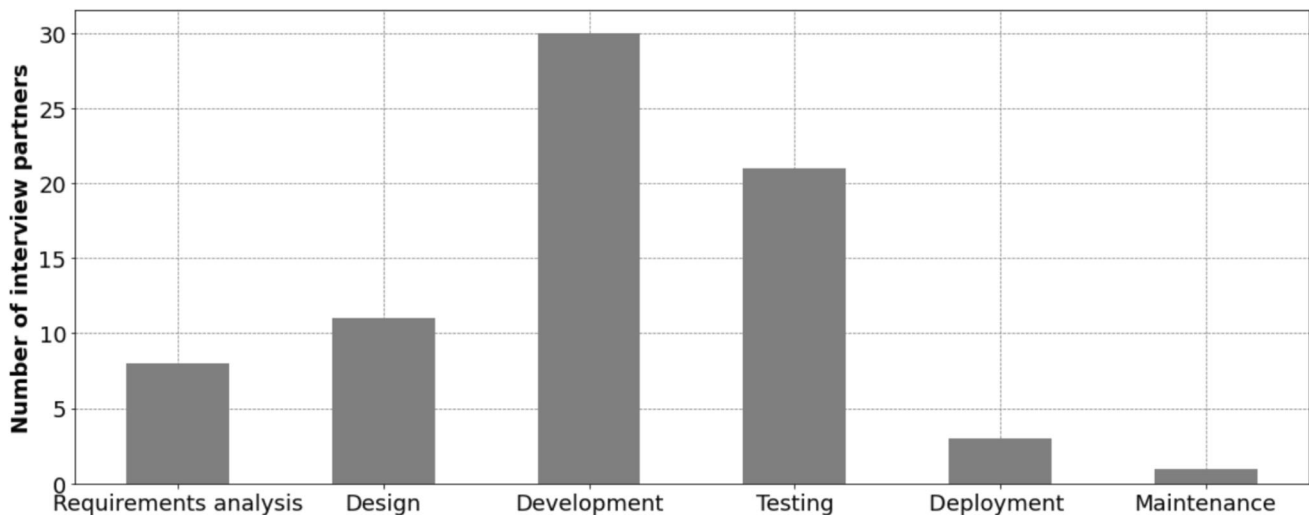
#### 4.7 Applications Within the Software Development Life Cycle

After examining how GenAI's functional affordances contribute to the observed effects on individuals and organizations within the software development domain, we now provide a descriptive analysis investigating the extent to which the different SDLC phases are affected. To this end, we refer to the SDLC framework consisting of the stages of requirements analysis, design, development, testing, deployment, and maintenance. In the first step of our analysis, we identify the SDLC stages where practitioners explicitly mention to use or plan to use GenAI in the near future. Figure 3 presents the summary of GenAI usage across these stages.

All 30 interview partners explicitly mention the development phase as a current or upcoming application field for

GenAI systems and emphasize the application of GenAI systems for repetitive, time-consuming tasks that require limited conceptual skills. Another widespread application field, mentioned by 21 interview partners, is the testing phase. Many experts explain that they regularly use GenAI to facilitate the creation of software test cases or plan to do so. Eight experts mention the phase of requirements analysis as a suitable application field for GenAI systems. In this context, the experts use GenAI for both the improvement of existing requirements formulations and for brainstorming new, innovative requirements. Eleven experts use or plan to use GenAI in the design phase of software systems. In the later phases of the SDLC, which are deployment and maintenance, our experts rarely report any application fields. Three experts mention the deployment phase and only one expert explicitly mentions the maintenance of software products. To better understand why GenAI is applied unevenly across the SDLC, we next outline how the functional affordances relate to the activities performed in each phase.

In Table 2, we show not only the relationships between functional affordances and observed effects, but also the SDLC phases that we consider to be most affected. Several patterns stand out: The broad impact of the affordances *on-demand knowledge acquisition* and *organizational knowledge exchange* across all phases, the relatively narrow impact of most other affordances on a few phases only, and the tendency that cognitively stimulating affordances contribute disproportionately to the design phase. First, *on-demand knowledge acquisition* and *organizational knowledge exchange* have a broad impact on nearly all phases of the SDLC. The rationale seems straightforward: Faster access to knowledge, whether through explanations of unfamiliar concepts or efficient retrieval of internal information, is helpful throughout the SDLC because uncertainties arise in every phase. Second, most other affordances show effects that are specific to one or few phases of the SDLC only. The affordance of *creative exploration*, for example, is most relevant in the design phase, where developers benefit from support in brainstorming ideas and architectural designs for the planned software applications. Similarly, the affordance of *generation of code and documentation* primarily influences the development phase, where developers use generated code as an initial draft for their work. Third, we observe that the affordances that primarily stimulate developers' cognitive engagement have a disproportionately large influence on the design phase. These affordances bring fewer risks for individual learning and skill development but are more likely to reduce the intensity of collaboration between developers (see Sect. 4.6). As a result, we expect that the decline in peer interaction is most pronounced during the



**Fig. 3** Reported application fields of GenAI within SDLC

design phase, where developers engage in the cognitively stimulating tasks most intensely.

## 5 Discussion

In this section, we discuss our findings and their implications for theory and practice. We begin with an interpretation of our results, followed by a theoretical integration covering the streams of social capital, machine-induced learning, and professional identity. We then outline practical implications before addressing limitations and future research opportunities.

### 5.1 Interpretation of Results

In this study, we conducted 30 expert interviews to analyze the sociotechnical impact of GenAI on software developers and IT organizations from the perspective of software development practitioners. In line with recent literature on GenAI (e.g., Pothukuchi et al. 2023), we find that developers deem GenAI to be highly impactful in several phases of the SDLC, most notably in development and testing. Interestingly, a smaller fraction of the interviewed experts reports current usage of GenAI in the requirements analysis and design phases, while almost no current applications are mentioned for deployment and maintenance. We interpret these findings in light of the current maturity of both GenAI technology and its integration into organizations. Regarding GenAI maturity, at the time of our data collection, GenAI systems were mostly implemented as chatbots. As GenAI advances and becomes increasingly integrated into AI agents (c.f. Wang et al. 2024b; Acharya et al. 2025), enabling it to autonomously use development

tools such as debuggers, its applications across all stages of the SDLC is expanding. For example, recent agentic coding systems such as Anthropic's Claude Code, which can autonomously generate, structure, and iteratively refine substantial parts of software projects, illustrate the potentially disruptive implications of this technological trajectory for software development work and associated labor markets (Metz 2026). Regarding the maturity of GenAI integration, many organizations at the time of data collection lacked the necessary conditions to fully leverage GenAI. As our interviewed experts emphasized, frameworks for rigorous error correction, careful data protection, and employee training programs are essential for effective GenAI adoption. As both GenAI technology and its organizational integration mature, we can expect its use within the SDLC to become significantly more widespread.

Our interviews reveal both opportunities and pitfalls of GenAI from the perspective of software developers. With regard to opportunities, experts emphasize productivity gains, particularly in well-defined coding tasks, documentation, and other technical writing activities. These findings suggest that the productivity gains observed in existing experimental studies (Peng et al. 2023; Nam et al. 2024) do indeed translate into real-world software development environments. Beyond productivity, our findings suggest that GenAI can serve as a valuable tool for skill development and self-education. This learning support is especially beneficial for junior developers, helping them acquire technical skills such as new programming languages and coding practices. Along with these training opportunities, our experts expect shifts in the talent market for software developers. As skill barriers for basic programming skills fade, our interviewees expect a rise in the supply of and competition between junior-level developers. On the other

hand, the demand for senior-level developers is expected to grow significantly as GenAI does not help junior or mid-level developers to rapidly acquire the advanced conceptual and management skills needed to transition into senior roles. Interestingly, developers working in a start-up environment emphasized that GenAI enabled them to take on broader responsibilities by engaging more flexibly with different projects and technologies, thereby reducing the need for additional hiring. While we cannot conclude from our evidence that this dynamic is specific to start-ups, it may explain the declining demand for junior developers in organizational settings where resources are limited, while further reinforcing the importance of senior developers for oversight and coordination. Another important opportunity is GenAI's potential to stimulate creativity. While GenAI itself does not generate fundamentally novel ideas, developers describe its role as a "sparring partner" (I21, Quality manager; I24, Data scientist) that challenges their current thinking and broadens their creative problem-solving abilities.

Whether productivity gains from technology on the individual level translate into increased organizational productivity is far from straightforward. Prior research shows that although new technology often brings individual benefits, inefficiencies in processes such as organizational bottlenecks may still limit organizational productivity; a phenomenon commonly referred to as the "IT productivity paradox" (Brynjolfsson 1993). Behind this background, our interviews reveal that the cumulative effects of individual productivity improvements indeed allow organizations to take on a higher number of projects without proportionally increasing (human) resources. Furthermore, we identify process streamlining and standardization as a second channel that contributes to organizational productivity beyond individual productivity improvements. Regarding the effects on human-to-human collaboration, experts have ambivalent attitudes: Some experts highlight that developers gain quick access to valuable information, saving unnecessary communication between employees; other experts view this reduction of communication between developers as problematic because it may inhibit the development of interpersonal relationships and, consequently, team cohesion. A weakened team cohesion is one of many organizational pitfalls voiced by the interviewed experts, others refer to poor engagement with GenAI-generated output, which may reduce software quality.

One potential pitfall that we consider particularly worthy of discussion is that excessive reliance on GenAI may erode developers' coding routines. We interpret these routines as developers' unique programming patterns and standardized approaches to problem-solving (Parnin et al. 2017; Masood et al. 2022) and recognize two ways in

which this pitfall can impact organizations. First, developers may gradually lose their programming patterns and expertise and, as a consequence, overrely on GenAI. An overreliance on GenAI can be particularly problematic because it may go unnoticed through a GenAI-induced "illusion of competence" (Prather et al. 2024) – or referred to as "Dunning-Kruger effect" by several of our interviewees – so that flawed or entirely hallucinated AI outputs may go unchallenged and ultimately compromise software quality. Second, rather than losing their coding routines entirely, developers' coding may converge towards a common, GenAI-influenced approach. However, research has repeatedly shown that diversity in programming patterns and expertise within teams is crucial for fostering creativity and innovation (e.g., Tiwana and McLean 2005). A shift toward homogenized coding routines could thus reduce the breadth of problem-solving strategies in organizations. While both issues seem severe for organizations, one might alternatively argue that an erosion of coding routines is simply a reflection of an ongoing evolution in software development. Over time, programming has moved from machine code to high-level languages and, more recently, to low-code and no-code environments (Al Alamin et al. 2021). This shift has allowed developers to focus on higher-level problem-solving rather than low-level implementation details. However, a crucial distinction remains: low-code and no-code environments can still be deterministic in nature and require developers to engage in algorithmic thinking (Matook et al. 2025). By contrast, GenAI shifts the developer's role from algorithmic problem-solving to co-creation with an AI partner, to whom much of the thought process can be transferred. We caution that this shift may lead to the aforementioned decline of programming expertise.

Developers use GenAI systems not only for obvious purposes such as autocompletion and code snippet generation. Instead, we identify seven different functional affordances, each of which presents different opportunities and pitfalls for software developers and their organizations. Notably, as outlined in Sect. 2.1, these affordances are in many ways unique to GenAI as an underlying technology. For instance, affordances such as code review existed prior to the rise of GenAI through deterministic predecessor tools but have reached an entirely new level of sophistication with GenAI. Others, such as creative exploration, were not feasible at all before the advent of GenAI (Feuerriegel et al. 2024). While some affordances, particularly the outsourcing of tedious coding work, bring pitfalls primarily for the professional growth of developers, other affordances partially replace team interactions and, thus, may negatively affect organizations' collaborative culture and team cohesion. What stands out is the affordance of organizational knowledge exchange for which our data do

not imply any associated pitfall. We find this worth discussing as we do conjecture possible pitfalls not for the organizational level, but for individual employees. Especially for experienced employees, possessing unique organizational knowledge, such as knowing where critical information resides, how to navigate legacy systems, or whom to approach for specific problems, represents a competitive advantage (Černe et al. 2014). If GenAI makes internal knowledge broadly accessible, this type of informational gatekeeping can vanish. While desirable at the organizational level, this would reduce the bargaining power and competitive advantages of individuals whose distinctiveness rested on hard-to-access knowledge. Although our data do not show such concerns among our interviewed practitioners, we find such effects worth investigating in future work.

Overall, our findings demonstrate how GenAI marks a fundamental shift in the relationship between humans and IT artifacts, that is, from a “primacy to human agency” (Baird and Maruping 2021) toward agentic artifacts that co-create software alongside human developers (Feuerriegel et al. 2024). This transition brings with it both substantial opportunities and risks. Our study contributes to a deeper understanding of this paradigm shift and its sociotechnical implications for software development.

## 5.2 Theoretical Integration

Our study builds on and contributes to several streams of literature (as outlined in Sect. 2). In the following, we detail how our findings contribute to the literature on social capital in organizations, on machine-induced learning and reflections, and on professional identity.

### 5.2.1 Social Capital in Organizations

Our findings highlight several ways in which GenAI shapes social capital within organizations. As detailed in Sect. 2.3, social capital in organizations relies on organizational citizenship behavior (Organ 1988), which emerges through daily interactions among employees, i.e., software developers. These interactions often involve colleagues assisting each other with technical challenges, such as bug fixing. However, our findings suggest that as GenAI increasingly assumes the role of a “helpful citizen”, such interactions between human colleagues may decline, potentially weakening social capital within organizations. This insight is particularly significant given that prior research has primarily emphasized the positive impact of technology on social capital, such as by enabling network infrastructure and tools that facilitate such organizational citizenship behavior (Huysman and Wulf 2006; Yang et al. 2009). Importantly, social capital plays a crucial role in bridging

gaps in individual skills and knowledge, serving as a key enabler of software development in organizations (Šmite et al. 2017). Its weakening, therefore, may pose a substantial organizational challenge.

By contrast, our findings also support the view that GenAI does not necessarily erode social capital and may even contribute to its accumulation. In response to the decline in interactions and a potential reduction in organizational citizenship behaviors, developers may adapt by finding new ways to network and build social capital. This adaptability is supported by prior research showing that social capital and knowledge networks in software development can take various forms, even within the same organization (Šmite et al. 2017). Moreover, GenAI may enhance social capital by improving communication quality among developers. Specifically, it allows developers to seek definitions or clarify concepts before consulting colleagues, leading to more informed and productive discussions. In this way, GenAI aligns with previous research demonstrating that technology can support communication and, in turn, foster organizational citizenship behavior.

Overall, our findings indicate that GenAI will influence social capital in the long term, possibly in both positive and negative ways. This complements the emerging literature on how technology adoption shapes social capital in organizations and highlights the need for further exploration.

### 5.2.2 Machine-Induced Learning and Reflection

Our findings contribute to the growing stream of literature on machine-induced learning and reflection (see Sect. 2.4). Our results show that developers view GenAI as a “sparing partner” (I21, Quality manager; I24, Data scientist); that is, they report using it to explore alternative solutions, self-educate, and creatively enhance their own problem-solving. We interpret this as evidence that GenAI fosters reflective thinking in software development. This interpretation aligns with theory proposed by Pfeuffer et al. (2023), who suggest that AI giving diverging predictions strongly stimulates reflection in professionals by triggering a “challenge mode.” In this mode, individuals attempt to reconcile their initial judgments with the AI’s output, promoting deeper cognitive engagement and reflection. Similarly, developers in our study appeared to engage in such reconciliation when faced with suggestions and alternative solutions proposed by GenAI, providing support for the idea that machine-induced reflections extend from regression and classification tasks to generative AI.

Our interpretation of diverging GenAI output prompting reflective thinking among developers aligns with the concept of a cognitive disconfirmation pattern as proposed by Jussupow et al. (2021). Jussupow et al. (2021) identify

various patterns of how professionals respond to AI outputs that differ from their own assessments. These patterns include reflecting on the task at hand, reassessing one's own beliefs, assessing how the AI arrived at a diverging output, or a combination of these, which is referred to as "active consideration". Our interviews highlight not only GenAI's potential to foster reflective thinking but also the need for rigorous evaluation of GenAI outputs to identify errors – both of which can potentially be addressed through developers engaging in such "active consideration" of both their own and the AI's assessments.

### 5.2.3 Professional Identity

Our study contributes to the nascent literature on the role of AI for professional identity in the workplace (e.g., Strich et al. 2021; Jussupow et al. 2022; Mirbabaie et al. 2022). As detailed in Sect. 2.5, employees can become strongly attached to and identify with their profession (Pratt et al. 2006; Stein et al. 2013). AI may threaten this professional identity, as employees fear being replaced (Mirbabaie et al. 2022) or challenged in their status as a knowledgeable and valued expert (Jussupow et al. 2022). Our study extends this line of work by showing that GenAI may threaten the expertise monopoly of a profession previously unchallenged by AI, namely software development. Coding expertise, once a defining skill, is now increasingly replaceable by GenAI. According to existing theory, such an identity threat would likely lead software developers to resist the introduction of GenAI (Mirbabaie et al. 2022). However, none of our interviewees report such resistance. This could be due to software developers' generally high affinity for and openness to new technology, aligning with Craig et al. (2019), who argue that individual characteristics play a crucial role in determining whether and to what extent identity threats from new technology translate into resistance.

Our findings further show that developers confronted with the availability of new GenAI systems tend to adjust their self-perception, that is, change their professional identity. Following our findings, GenAI may prompt a shift in identity from coders to system orchestrators. While this shift may pose challenges for developers, the literature suggests that they will eventually succeed in reshaping their professional identity as they gain more experience with GenAI (e.g., Lamb and Davidson 2005; Korica and Molloy 2010; Stein et al. 2013). Moreover, junior developers entering the job market may anticipate this looming identity shift and adjust their career aspirations accordingly. Indeed, such behavior aligns with theoretical and empirical insights from other domains. For instance, Jussupow et al. (2022) found that medical students who expected AI to automate aspects of the physician's role

were less likely to pursue specialties vulnerable to automation, such as radiology. Likewise, aspiring software developers may prioritize roles that are less susceptible to GenAI substitution, potentially accelerating the shift in professional identity they seek to navigate.

### 5.3 Practical Implications

Our study reveals important practical implications for IT organizations. While the opportunities of GenAI in software development are substantial, organizations must actively manage its pitfalls to ensure a sustainable benefit. In the following, we discuss possible strategies to address these pitfalls along four areas: skill development, preservation of social capital, recognition of new professional identities, and quality assurance through rigorous governance.

One central concern voiced by our interviewees was the risk of developers becoming overly dependent on GenAI, potentially leading to skill degradation. As a remedy, organizations could adopt interventions to maintain and foster core competencies. One possible approach is to introduce manual coding sessions periodically, in which developers deliberately complete coding exercises or sprints without GenAI support. These sessions would not only help preserve problem-solving abilities but can also serve as informal skill checks, providing insights that can guide the design of targeted training programs.

Our findings further indicate that GenAI may replace peer-to-peer assistance on coding issues, potentially weakening social connections and opportunities for informal knowledge exchange. To mitigate this risk, organizations could implement deliberate practices that maintain interpersonal ties. Examples include mandatory peer reviews, designated office hours where junior developers can (and are encouraged to) approach senior colleagues, and collaborative coding challenges with a focus on pair programming. Such practices ensure that GenAI complements rather than replaces social interactions within teams.

Our interviews also reveal an upcoming shift in the professional identity of developers from coders toward more hybrid roles and "orchestrators" of GenAI systems. Such identity shifts are often accompanied by tensions and self-doubt (Jussupow et al. 2018) which stresses the need for organizations to actively acknowledge and validate this new role of software developers. Job descriptions and evaluation systems should be adjusted to show that these skills are not minor extras but central to modern software development workflows. In addition, mentorship and training programs should help developers gain confidence in these new roles and see their evolving expertise as an organizational asset rather than a threat.

Finally, to alleviate concerns associated with GenAI regarding data protection, software quality, and maintainability, organizations need to establish clear governance structures. First, they should maintain a list of approved tools and define where and how these tools can be used, as well as areas where GenAI use is prohibited entirely (e.g., highly sensitive code bases). Second, organizations should define clear escalation procedures to ensure developers know how to respond when GenAI tools fail or generate problematic outputs. Third, for code quality assurance, organizations should ensure that all GenAI-generated code is rigorously reviewed before entering production, using both automated tests and human code reviews. Finally, monitoring error rates and the effort required to correct GenAI-generated code over time can help organizations assess whether its use ultimately improves or undermines software quality.

#### 5.4 Limitations and Future Work

While this study provides valuable insights into the perceptions and attitudes of software development practitioners toward GenAI, we acknowledge several limitations. First, the global landscape of GenAI is rapidly evolving, with new technologies and practices emerging at a fast pace. This highly dynamic environment requires the research community to continuously investigate the impact of new technologies on software development. For example, as GenAI becomes more powerful and increasingly integrated into agentic coding systems (c.f. Acharya et al. 2025), it evolves from supporting or completing tasks within the SDLC towards actively managing and coordinating the software project itself (see, e.g., Anthropic's Claude Code). Future research should therefore continue engaging in this field and analyze the sociotechnical impact over time. Second, and related to the previous point, our data on how GenAI's functional affordances shape opportunities and pitfalls across different phases of the SDLC should be interpreted as suggestive rather than conclusive evidence. This aligns with our earlier call for continued investigation into how the effects of GenAI unfold across SDLC phases and how these dynamics evolve as the technology matures. Third, we intentionally designed our interview guideline with a broad perspective, covering multiple aspects of developers' perceptions toward GenAI. However, this holistic interview approach limited the amount of time we could spend on each specific aspect. Therefore, we call for future research to focus on single aspects, such as the impact of GenAI on collaboration dynamics within software development teams, to gain a deeper understanding of these specific concepts. Finally, our findings reflect subjective impressions from developers and do not establish causal mechanisms in the way that, for

instance, a randomized field experiment would. Following Klein and Myers (1999), we propose empirically grounded theoretical relationships within a conceptual model. This model should be viewed as a springboard for future (quantitative) research to more closely examine the causal mechanisms at play.

## 6 Conclusion

Our study provides valuable insights into the sociotechnical impact of GenAI from the perspective of software development practitioners. The identified application fields of GenAI within the SDLC framework indicate potential for deeper integration of these technologies into software development projects. Further, our findings reveal both the opportunities and pitfalls of GenAI, highlighting the need for well-considered implementation strategies to boost productivity while safeguarding against potential side effects such as skill degradation and over-reliance on AI systems. Thus, our research highlights the importance of a balanced approach that reaps the benefits of GenAI while fostering an environment that supports the professional growth of software developers. Future research should explore how these dynamics evolve over time and further investigate strategies to integrate GenAI in a manner that maximizes its positive impact while effectively managing associated side effects.

**Funding** Open Access funding enabled and organized by Projekt DEAL.

**Supplementary Information** The online version contains supplementary material available at <https://doi.org/10.1007/s12599-026-00998-y>.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

## References

- Abdel-Karim BM, Pfeuffer N, Carl KV, Hinz O (2023) How AI-based systems can induce reflections: the case of ai-augmented diagnostic work. *MIS Q* 47(4)
- Abouei Ardakan M, Sima M, Zeinal Hamadani A, Coit DW (2016) A novel strategy for redundant components in reliability-redundancy allocation problems. *IEEE Trans* 48(11):1043–1057
- Acharya DB, Kuppan K, Divya B (2025) Agentic ai: autonomous intelligence for complex goals – a comprehensive survey. *IEEE Access*
- Adler PS, Kwon SW (2002) Social capital: prospects for a new concept. *Acad Manag Rev* 27(1):17–40
- Al Alamin MA, Malakar S, Uddin G, Afroz S, Haider TB, Iqbal A (2021) An empirical study of developer discussions on low-code software development challenges. In: 2021 IEEE/ACM 18th international conference on mining software repositories (msr). IEEE, pp 46–57
- Ammari G, Alkurdi B, Alshurideh A, Alrowwad A (2017) Investigating the impact of communication satisfaction on organizational commitment: a practical approach to increase employees' loyalty. *Int J Mark Stud* 9(2):113–133
- Armbrust O, Katahira M, Miyamoto Y, Münch J, Nakao H, Ocampo A (2009) Scoping software process lines. *Softw Process Improve Pract* 14(3):181–197
- Baird A, Maruping LM (2021) The next generation of research on is use: a theoretical framework of delegation to and from agentic is artifacts. *MIS Q* 45(1)
- Banh L, Strobel G (2023) Generative artificial intelligence. *Electron Mark* 33(1):63
- Banh L, Holldack F, Strobel G (2025) Copiloting the future: How generative ai transforms software engineering. *Inf Softw Technol* 183(107):751
- Banker RD, Kauffman RJ (1991) Reuse and productivity in integrated computer-aided software engineering: an empirical study. *MIS Q* 375–401
- Bassellier G, Reich BH, Benbasat I (2001) Information technology competence of business managers: a definition and research model. *J Manag Inf Syst* 17(4):159–182
- Bassil Y (2012) A simulation model for the waterfall software development life cycle. [arXiv:1205.6904](https://arxiv.org/abs/1205.6904)
- Bauer K, von Zahn M, Hinz O (2023) Expl(AI)ned: the impact of explainable artificial intelligence on users' information processing. *Inf Syst Res* 34(4):1582–1602
- Benlian A, Pinski M, Adam M (2025) Team-enacted use versus developer-needed use of agile practices: How perceptual (in-)congruence and team feedback-seeking shape developer well-being. *Inf Syst Res*
- Bolino MC, Turnley WH, Bloodgood JM (2002) Citizenship behavior and the creation of social capital in organizations. *Acad Manag Rev* 27(4):505–522
- Bosnjak M, Ajzen I, Schmidt P (2020) The theory of planned behavior: selected recent advances and applications. *Eur J Psychol* 16(3):352
- Bostrom RP, Gupta S, Thomas D (2009) A meta-theory for understanding information systems within sociotechnical systems. *J Manag Inf Syst* 26(1):17–48
- Brooks NG, Hardgrave BC, O'Leary-Kelly AM, McKinney V, Wilson DD (2015) Identifying with the information technology profession: implications for turnaway of it professionals. *ACM SIGMIS Database* 46(1):8–23
- Brynjolfsson E (1993) The productivity paradox of information technology. *Commun ACM* 36(12):66–77
- Carzaniga A, Gorla A, Pezzè M (2009) Handling software faults with redundancy. *Architecting dependable systems VI*. Springer, Heidelberg, pp 148–171
- Cascella M, Montomoli J, Bellini V, Bignami E (2023) Evaluating the feasibility of chatgpt in healthcare: an analysis of multiple clinical and research scenarios. *J Med Syst* 47(1):33
- Černe M, Nerstad CG, Dysvik A, Škerlavaj M (2014) What goes around comes around: knowledge hiding, perceived motivational climate, and creativity. *Acad Manag J* 57(1):172–192
- Cerullo M (2024) Tech companies are slashing thousands of jobs as they pivot toward ai. <https://www.cbsnews.com/news/tech-lay-offs-artificial-intelligence-ai-chatgpt/>. Accessed 01 Jan 2025
- Chang Y, Wang X, Wang J, Wu Y, Yang L, Zhu K, Chen H, Yi X, Wang C, Wang Y, Ye W, Zhang Y, Chang Y, Yu PS, Yang Q, Xie X (2024) A survey on evaluation of large language models. *ACM Trans Intell Syst Technol* 15(3):39:1–39:45. <https://doi.org/10.1145/3641289>
- Chatterjee S, Sarker S, Lee MJ, Xiao X, Elbanna A (2021) A possible conceptualization of the information systems (is) artifact: a general systems theory perspective 1. *Inf Syst J* 31(4):550–578
- Chen T (2024) The impact of ai-pair programmers on code quality and developer satisfaction: evidence from timi studio. In: Proceedings of the 2024 international conference on generative artificial intelligence and information security, pp 201–205
- Clopton AW (2011) Social capital and team performance. *Team Perform Manag* 17(7/8):369–381
- Craig K, Thatcher JB, Grover V (2019) The it identity threat: a conceptual definition and operational measure. *J Manag Inf Syst* 36(1):259–288
- Deniz BK, Gnanasambandam C, Harrysson M, Hussin A, Srivastava S (2023) Unleashing developer productivity with generative AI. <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai>. Accessed 01 Jan 2025
- Devi S, Garg H, Garg D (2023) A review of redundancy allocation problem for two decades: bibliometrics and future directions. *Artif Intell Rev* 56(8):7457–7548
- Dhake SP, Lassi L, Hippalgaonkar A, Gaidhani RA, NM J (2024) Impacts and implications of generative AI and large language models: redefining banking sector. *J Inform Educ Res* 4(2)
- Ebert C, Louridas P (2023) Generative ai for software practitioners. *IEEE Softw* 40(4):30–38
- Etikan I, Musa SA, Alkassim RS et al (2016) Comparison of convenience sampling and purposive sampling. *Am J Theor Appl Stat* 5(1):1–4
- Feuerriegel S, Hartmann J, Janiesch C, Zschech P (2024) Generative AI. *Bus Inf Syst Eng* 66(1):111–126
- Forsgren N, Storey MA, Maddila C, Zimmermann T, Houck B, Butler J (2021) The space of developer productivity. *Commun ACM* 64(6):46–53
- Gioia DA, Corley KG, Hamilton AL (2013) Seeking qualitative rigor in inductive research: notes on the gioia methodology. *Organ Res Methods* 16(1):15–31
- Glaser B, Strauss A (2017) Discovery of grounded theory: strategies for qualitative research. Routledge, London
- Graziotin D, Fagerholm F, Wang X, Abrahamsson P (2018) What happens when software developers are (un) happy. *J Syst Softw* 140:32–47
- Hoyek ME, Müller N, Ronellenfisch J (2024) Gen ai's productivity promise: Huge potential but most have not yet reached scaled impact. <https://www.mckinsey.com/capabilities/operations/our-insights/operations-blog/gen-ais-productivity-promise-huge-potential-but-most-have-not-yet-reached-scaled-impact>. Accessed 01 Jan 2025

- Huysman M, Wulf V (2006) It to support knowledge sharing in communities, towards a social capital analysis. *J Inf Technol* 21:40–51
- Imai S (2022) Is github copilot a substitute for human pair-programming? an empirical study. In: *Proceedings of the acm/ieee 44th international conference on software engineering: companion proceedings*, pp 319–321
- Jemielniak D (2008) Software engineers or artists? programmers' identity choices. *Tamara J Crit Organ Inq* 7(1)
- Joshi KD, Sarker S, Sarker S (2007) Knowledge transfer within information systems development teams: examining the role of knowledge source attributes. *Decis Support Syst* 43(2):322–335
- Jürgensmeier L, Skiera B (2024) Generative AI for scalable feedback to multimodal exercises. *Int J Res Mark* 41(3):468–488
- Jussupow E, Spohrer K, Heinzl A, Link C (2018) I am; we are – conceptualizing professional identity threats from information technology. In: *International conference of information systems (ICIS)*
- Jussupow E, Spohrer K, Heinzl A, Gawlitza J (2021) Augmenting medical diagnosis decisions? An investigation into physicians' decision-making process with artificial intelligence. *Inf Syst Res* 32(3):713–735
- Jussupow E, Spohrer K, Heinzl A (2022) Identity threats as a reason for resistance to artificial intelligence: survey study with medical students and professionals. *JMIR Form Res* 6(3):e28750
- Klein HK, Myers MD (1999) A set of principles for conducting and evaluating interpretive field studies in information systems. *MIS Q* 67–93
- Korica M, Molloy E (2010) Making sense of professional identities: stories of medical professionals and new technologies. *Hum Relat* 63(12):1879–1901
- Lamb R, Davidson E (2005) Information and communication technology challenges to scientific professional identity. *Inf Soc* 21(1):1–24
- Lee D, Yoon SN (2021) Application of artificial intelligence-based technologies in the healthcare industry: opportunities and challenges. *Int J Environ Res Public Health* 18(1):271
- Li H, Su J, Chen Y, Li Q, ZHANG ZX (2024) Sheetcopilot: bringing software productivity to the next level through large language models. *Adv Neural Inf Process Syst* 36
- Liebowitz J, Yan C (2004) Knowledge sharing proficiencies: the key to knowledge management. In: *Handbook on knowledge management 1: knowledge matters*, pp. 409–424
- Madaio M, Egede L, Subramonyam H, Wortman Vaughan J, Wallach H (2022) Assessing the fairness of ai systems: Ai practitioners' processes, challenges, and needs for support. *Proc ACM Hum-Comput Interact* 6(CSCW1):1–26
- Marks A, Scholarios D (2007) Revisiting technical workers: professional and organisational identities in the software industry. *New Technol Work Employ* 22(2):98–117
- Markus ML, Silver MS (2008) A foundation for the study of IT effects: a new look at desanctis and poole's concepts of structural features and spirit. *J Assoc Inf Syst* 9(10):5
- Masood Z, Hoda R, Blincoc K, Damian D (2022) Like, dislike, or just do it? how developers approach software development tasks. *Inf Softw Technol* 150(106):963
- Matook S, Wang YM, Axelsen M (2025) Experiential learning for citizen developers. *Bus Inf Syst Eng* 1–24
- Melville NP, Robert L, Xiao X (2023) Putting humans back in the loop: an affordance conceptualization of the 4th industrial revolution. *Inf Syst J* 33(4):733–757
- Metz C (2016) The A.I. that writes code is reshaping tech jobs. *The New York Times Magazine*. <https://www.nytimes.com/2026/03/12/magazine/ai-coding-programmingjobs-claude-chatgpt.html>
- Mirbabaie M, Brünker F, Möllmann NR, Stieglitz S (2022) The rise of artificial intelligence—understanding the ai identity threat at the workplace. *Electron Mark* 32:1–27
- Mumford E (2006) The story of socio-technical design: reflections on its successes, failures and potential. *Inf Syst J* 16(4):317–342
- Myers MD, Newman M (2007) The qualitative interview in is research: examining the craft. *Inf Organ* 17(1):2–26
- Nam D, Macvean A, Hellendoorn V, Vasilescu B, Myers B (2024) Using an llm to help with code understanding. In: *2024 IEEE/ACM 46th international conference on software engineering (ICSE)*. IEEE Computer Society, pp 881–881
- Negrin KA, Slaughter SE, Dahlke S, Olson J (2022) Successful recruitment to qualitative research: a critical reflection. *Int J Qual Methods* 21(16094069221119):576
- Noy S, Zhang W (2023) Experimental evidence on the productivity effects of generative artificial intelligence. *Sci* 381(6654):187–192
- Organ DW (1988) *Organizational citizenship behavior: The good soldier syndrome*. Lexington books/DC heath and com
- Orlikowski WJ (1993) Case tools as organizational change: Investigating incremental and radical changes in systems development. *MIS Q* 309–340
- O'Connor C, Joffe H (2020) Intercoder reliability in qualitative research: debates and practical guidelines. *Int J Qual Methods* 19(1609406919899):220
- Parnin C, Siegmund J, Peitek N (2017) On the nature of programmer expertise. In: *Ppig*, 16
- Peng S, Kalliamvakou E, Cihon P, Demirer M (2023) The impact of ai on developer productivity: evidence from github copilot. [arXiv:2302.06590](https://arxiv.org/abs/2302.06590)
- Pothukuchi AS, Kota LV, Mallikarjunaradhya V (2023) Impact of generative ai on the software development lifecycle (sdhc). *Int J Creat Res Thoughts* 11(8)
- Prather J, Reeves BN, Leinonen J, MacNeil S, Randrianasolo AS, Becker BA, Kimmel B, Wright J, Briggs B (2024) The widening gap: the benefits and harms of generative AI for novice programmers. In: *Proceedings of the 2024 acm conference on international computing education research-volume 1*, pp 469–486
- Pratt MG, Rockmann KW, Kaufmann JB (2006) Constructing professional identity: the role of work and identity learning cycles in the customization of identity among medical residents. *Acad Manag J* 49(2):235–262
- Ramaul L, Ritala P, Ruokonen M (2024) Creational and conversational AI affordances: how the new breed of chatbots is revolutionizing knowledge industries. *Bus Horiz* 67(5):615–627
- Reagans R, Zuckerman EW (2001) Networks, diversity, and productivity: the social capital of corporate r&d teams. *Organ Sci* 12(4):502–517
- Riemenschneider CK, Armstrong DJ (2021) The development of the perceived distinctiveness antecedent of information systems professional identity. *MIS Q* 45(3)
- Ross SI, Martinez F, Houde S, Muller M, Weisz JD (2023) The programmer's assistant: conversational interaction with a large language model for software development. In: *Proceedings of the 28th international conference on intelligent user interfaces*, 491–514
- Ryan SD, Harrison DA, Schkade LL (2002) Information-technology investment decisions: when do costs and benefits in the social subsystem matter? *J Manag Inf Syst* 19(2):85–127
- Sarker S, Xiao X, Beaulieu T, Lee AS (2018) Learning from first-generation qualitative approaches in the is discipline: an evolutionary view and some implications for authors and evaluators (part 1/2). *J Assoc Inf Syst* 19(8):752–774

- Sarker S, Chatterjee S, Xiao X, Elbanna A (2019) The sociotechnical axis of cohesion for the discipline: its historical legacy and its continued relevance. *MIS Q* 43(3):695–720
- Seibert SE, Kraimer ML, Liden RC (2001) A social capital theory of career success. *Acad Manag J* 44(2):219–237
- Šmite D, Moe NB, Šāblis A, Wohlin C (2017) Software teams and their knowledge networks in large-scale software development. *Inf Softw Technol* 86:71–86
- Söllner M, Arnold T, Benlian A, Bretschneider U, Knight C, Ohly S, Rudkowski L, Schreiber G, Wendt D (2025) Chatgpt and beyond: exploring the responsible use of generative AI in the workplace: an interdisciplinary perspective. *Bus Inf Syst Eng* 1–15
- Stein MK, Galliers RD, Markus ML (2013) Towards an understanding of identity and technology in the workplace. *J Inf Technol* 28(3):167–182
- Stein MK, Newell S, Wagner EL, Galliers RD (2015) Coping with information technology. *MIS Q* 39(2):367–392
- Stewart KJ, Gosain S (2006) The impact of ideology on effectiveness in open source software development teams. *MIS Q* 291–314
- Storey VC, Yue WT, Zhao JL, Lukyanenko R (2025) Generative artificial intelligence: evolving technology, growing societal impact, and opportunities for information systems research. *Inf Syst Front* 1–22. <https://doi.org/10.1007/s10796-025-10581-7>
- Strauss A, Corbin J (1998) Basics of qualitative research techniques. Citeseer
- Strich F, Mayer AS, Fiedler M (2021) What do I do in a world of artificial intelligence? Investigating the impact of substitutive decision-making AI systems on employees' professional role identity. *J Assoc Inf Syst* 22(2):304–324
- Suh S, An P (2022) Leveraging generative conversational ai to develop a creative learning environment for computational thinking. In: Companion proceedings of the 27th international conference on intelligent user interfaces, pp 73–76
- Susarla A, Gopal R, Thatcher JB, Sarker S (2023) The janus effect of generative ai: charting the path for responsible conduct of scholarly activities in information systems. *Inf Syst Res* 34(2):399–408
- Szulanski G (1996) Exploring internal stickiness: impediments to the transfer of best practice within the firm. *Strateg Manag J* 17(S2):27–43
- Thorbecke C (2023) AI is already linked to layoffs in the industry that created it. <https://edition.cnn.com/2023/07/04/tech/ai-tech-layoffs/index.html>. Accessed Jan 2025
- Tiwana A, McLean ER (2005) Expertise integration and creativity in information systems development. *J Manag Inf Syst* 22(1):13–43
- Ullsnes R, Moe NB, Stray V, Skarpen M (2024) Transforming software development with generative AI: empirical insights on collaboration and workflow. *Generative AI for effective software development*. Springer, Heidelberg, pp 219–234
- Wang W, Ning H, Zhang G, Liu L, Wang Y (2024a) Rocks coding, not development – a human-centric, experimental evaluation of LLM-supported SE tasks. [arXiv:2402.05650](https://arxiv.org/abs/2402.05650)
- Wang X, Li B, Song Y, Xu FF, Tang X, Zhuge M, Pan J, Song Y, Li B, Singh J et al (2024b) Opendevin: an open platform for ai software developers as generalist agents. [arXiv:2407.16741](https://arxiv.org/abs/2407.16741)
- Weiler M, Stolz S, Lanz A et al (2022) Social capital accumulation through social media networks: evidence from a randomized field experiment and individual-level panel data
- Wessel M, Adam M, Benlian A, Majchrzak A, Thies F (2025) Generative AI and its transformative value for digital platforms. *J Manag Inf Syst* 42(2):346–369. <https://doi.org/10.1080/07421222.2025.2487315>
- Windeler JB, Maruping L, Venkatesh V (2017) Technical systems development risk factors: the role of empowering leadership in lowering developers' stress. *Inf Syst Res* 28(4):775–796
- Xiang C, Lu Y, Gupta S (2013) Knowledge sharing in information system development teams: examining the impact of shared mental model from a social capital theory perspective. *Behav Inf Technol* 32(10):1024–1040
- Yan L, Greiff S, Teuber Z, Gašević D (2024) Promises and challenges of generative artificial intelligence for human learning. *Nat Hum Behav* 8(10):1839–1850
- Yang S, Lee H, Kurnia S (2009) Social capital in information and communications technology research: past, present, and future. *Commun Assoc Inf Syst* 25(1):23